- `BigInteger multiply(BigInteger other)`
- `BigInteger divide(BigInteger other)`
- `BigInteger mod(BigInteger other)`

  返回这个大整数和另一个大整数 other 的和、差、积、商以及余数。
- `BigInteger sqrt()` **9**

  得到这个 BigInteger 的平方根。
- `int compareTo(BigInteger other)`

  如果这个大整数与另一个大整数 other 相等，返回 0；如果这个大整数小于另一个大整数 other，返回负数；否则，返回正数。
- `static BigInteger valueOf(long x)`

  返回值等于 x 的大整数。

API java.math.BigDecimal 1.1

- `BigDecimal(String digits)`

  用给定数位构造一个大实数。
- `BigDecimal add(BigDecimal other)`
- `BigDecimal subtract(BigDecimal other)`
- `BigDecimal multiply(BigDecimal other)`
- `BigDecimal divide(BigDecimal other)`
- `BigDecimal divide(BigDecimal other, RoundingMode mode)` **5**

  返回这个大实数与另一个大实数 other 的和、差、积、商。如果商是一个无限小数，第一个 divide 方法会抛出一个异常。要得到一个舍入的结果，就要使用第二个方法。RoundingMode.HALF_UP 是我们在学校里学习的四舍五入方式（即，0 到 4 舍去，5 到 9 进位）。它适用于常规的计算。其他舍入方式请参见 API 文档。
- `int compareTo(BigDecimal other)`

  如果这个大实数与另一个大实数 other 相等，返回 0；如果这个大实数小于 other，返回负数；否则，返回正数。

3.10　数组

数组存储相同类型值的序列。下面几小节中，我们将学习在 Java 中如何使用数组。

3.10.1　声明数组

数组是一种数据结构，用来存储同一类型值的集合。通过一个整型*索引*（index，或称下标）可以访问数组中的每一个值。例如，如果 a 是一个整型数组，a[i] 就是数组中索引为 i 的整数。

在声明数组变量时，需要指出数组类型（元素类型后面紧跟 []）和数组变量名。例如，下面声明了整型数组 a：

```
int[] a;
```

不过，这条语句只声明了变量 a，并没有将 a 初始化为一个真正的数组。应该使用 new 操作符创建数组。

```
int[] a = new int[100]; // or var a = new int[100];
```

这条语句声明并初始化了一个可以存储 100 个整数的数组。

数组长度不要求是常量：new int[n] 会创建一个长度为 n 的数组。

一旦创建了数组，就不能再改变它的长度（不过，当然可以改变单个数组元素）。如果程序运行中需要经常扩展数组的大小，就应该使用另一种数据结构——数组列表（array list）。有关数组列表的详细内容请参见第 5 章。

注释： 可以使用下面两种形式定义一个数组变量：

```
int[] a;
```

或

```
int a[];
```

大多数 Java 程序员喜欢使用第一种风格，因为它可以将类型 int[]（整型数组）与变量名清晰地分开。

在 Java 中，提供了一种创建数组对象并同时提供初始值的简写形式。下面是一个例子：

```
int[] smallPrimes = { 2, 3, 5, 7, 11, 13 };
```

请注意，这个语法中不需要使用 new，甚至不用指定长度。

最后一个值后面允许有逗号，如果你要不断为数组增加值，这样会很方便：

```
String[] authors =
   {
      "James Gosling",
      "Bill Joy",
      "Guy Steele",
      // add more names here and put a comma after each name
   };
```

还可以声明一个*匿名数组*（anonymous array）：

```
new int[] { 17, 19, 23, 29, 31, 37 };
```

这个表达式会分配一个新数组并填入大括号中提供的值。它会统计初始值个数，并相应地设置数组大小。可以使用这种语法重新初始化一个数组而无须创建新变量。例如：

```
smallPrimes = new int[] { 17, 19, 23, 29, 31, 37 };
```

这是以下语句的简写形式：

```
int[] anonymous = { 17, 19, 23, 29, 31, 37 };
smallPrimes = anonymous;
```

注释：在 Java 中，允许有长度为 0 的数组。编写一个结果为数组的方法时，如果碰巧结果为空，这样一个长度为 0 的数组就很有用。可以如下构造一个长度为 0 的数组：

```
new elementType[0]
```

或

```
new elementType[] {}
```

注意，长度为 0 的数组与 null 并不相同。

3.10.2　访问数组元素

数组元素从 0 开始编号。最后一个合法的索引为数组长度减 1。在下面的例子中，索引值为 0 ～ 99。一旦创建了数组，就可以在数组中填入元素。例如，可以使用一个循环：

```
int[] a = new int[100];
for (int i = 0; i < 100; i++)
   a[i] = i; // fills the array with numbers 0 to 99
```

创建一个数字数组时，所有元素都初始化为 0。boolean 数组的元素会初始化为 false。对象数组的元素则初始化为一个特殊值 null，表示这些元素（还）未存放任何对象。初学者对此可能有些不解。例如，

```
String[] names = new String[10];
```

会创建一个包含 10 个字符串的数组，所有字符串都为 null。如果希望这个数组包含空串，则必须为元素提供空串：

```
for (int i = 0; i < 10; i++) names[i] = "";
```

警告：如果创建了一个包含 100 个元素的数组，然后试图访问元素 a[100]（或在 0 ～ 99 之外的任何其他索引），就会出现“array index out of bounds”（数组索引越界）异常。

要想获得数组中的元素个数，可以使用 array.length。例如，

```
for (int i = 0; i < a.length; i++)
   System.out.println(a[i]);
```

3.10.3　for each 循环

Java 有一种功能很强的循环结构，可以用来依次处理数组（或者任何其他元素集合）中的每个元素，而不必考虑指定索引值。

这种增强的 for 循环形式如下：

```
for (variable : collection) statement
```

它将给定变量（variable）设置为集合中的每一个元素，然后执行语句（statement）（当然，也可以是语句块）。collection 表达式必须是一个数组或者是一个实现了 Iterable 接口的类对象（例如 ArrayList）。有关数组列表的内容将在第 5 章中讨论，另外 Iterable 接口将在第 9 章中讨论。

例如，

```
for (int element : a)
   System.out.println(element);
```

会打印数组 a 的每一个元素，一个元素占一行。

这个循环应该读作“循环 a 中的每一个元素”(for each element in a)。Java 语言的设计者也曾考虑过使用诸如 foreach 和 in 这样的关键字，但这种循环并不是最初就包含在 Java 语言中，而是后来添加的，没有人希望破坏已经包含同名方法或变量（例如 System.in）的老代码。

当然，使用传统的 for 循环也可以获得同样的效果：

```
for (int i = 0; i < a.length; i++)
   System.out.println(a[i]);
```

但是，“for each”循环更加简洁、更不易出错，因为你不必为起始和终止索引值而操心。

注释：“for each”循环的循环变量将会遍历数组中的每个元素，而不是索引值。

如果需要处理一个集合中的所有元素，相比传统循环，“for each”循环是个让人欣喜的改进。不过，很多情况下还是需要使用传统的 for 循环。例如，可能不希望遍历整个集合，或者可能需要在循环内部使用索引值。

提示：有一个更容易的方法可以打印数组中的所有值，即利用 Arrays 类的 toString 方法。调用 Arrays.toString(a) 会返回一个包含数组元素的字符串，这些元素包围在中括号内，并用逗号分隔，例如，"[2,3,5,7,11,13]"。要想打印数组，只需要调用

```
System.out.println(Arrays.toString(a));
```

3.10.4 数组拷贝

在 Java 中，允许将一个数组变量拷贝到另一个数组变量。这时，两个变量将引用同一个数组：

```
int[] luckyNumbers = smallPrimes;
luckyNumbers[5] = 12; // now smallPrimes[5] is also 12
```

图 3-14 显示了拷贝的结果。

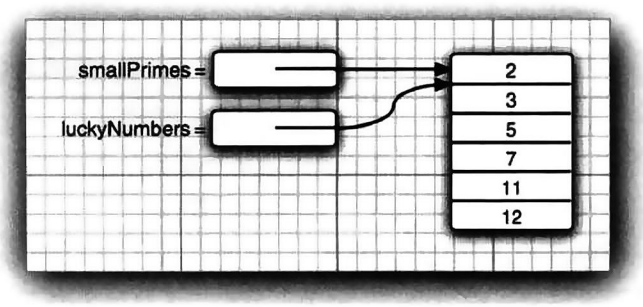


图 3-14 拷贝一个数组变量

如果确实希望将一个数组的所有值拷贝到一个新的数组中，就要使用 Arrays 类的 copyOf 方法：

```
int[] copiedLuckyNumbers = Arrays.copyOf(luckyNumbers, luckyNumbers.length);
```

第 2 个参数是新数组的长度。这个方法通常用来增加数组的大小：

```
luckyNumbers = Arrays.copyOf(luckyNumbers, 2 * luckyNumbers.length);
```

如果数组元素是数值型，那么新增的元素将填入 0；如果数组元素是布尔型，则填入 false。相反，如果长度小于原数组的长度，则只拷贝前面的值。

C++ 注释： Java 数组与堆栈上的 C++ 数组有很大不同，但基本上与在堆（heap）上分配的数组指针一样。也就是说，

```
int[] a = new int[100]; // Java
```

不同于

```
int a[100]; // C++
```

而等同于

```
int* a = new int[100]; // C++
```

Java 中的 [] 运算符预定义为会完成**越界检查**（bounds checking）。另外，没有指针运算，就意味着不能通过 a 加 1 得到数组中的下一个元素。

3.10.5　命令行参数

前面已经看到一个例子，其中一个 Java 数组重复出现过很多次。每一个 Java 程序都有一个带 String arg[] 参数的 main 方法。这个参数表明 main 方法将接收一个字符串数组，也就是命令行上指定的参数。

例如，来看下面这个程序：

```
public class Message
{
   public static void main(String[] args)
   {
      if (args.length == 0 || args[0].equals("-h"))
         System.out.print("Hello,");
      else if (args[0].equals("-g"))
         System.out.print("Goodbye,");
      // print the other command-line arguments
      for (int i = 1; i < args.length; i++)
         System.out.print(" " + args[i]);
      System.out.println("!");
   }
}
```

如果如下调用这个程序：

```
java Message -g cruel world
```

args 数组将包含以下内容：

```
args[0]: "-g"
args[1]: "cruel"
args[2]: "world"
```

这个程序会显示下面这个消息：

```
Goodbye, cruel world!
```

C++ 注释：在 Java 程序的 main 方法中，程序名并不存储在 args 数组中。例如，从命令行如下运行一个程序时

```
java Message -h world
```

args[0] 是 "-h"，而不是 "Message" 或 "java"。

3.10.6 数组排序

要想对数值型数组进行排序，可以使用 Arrays 类中的 sort 方法：

```
int[] a = new int[10000];
. . .
Arrays.sort(a)
```

这个方法使用了优化的快速排序（QuickSort）算法。快速排序算法对于大多数数据集都很高效。Arrays 类还提供了另外一些很便捷的方法，在这一节最后的 API 注释中将介绍这些方法。

程序清单 3-7 中的程序具体使用了数组，它会为一个抽彩游戏生成一个随机的数字组合。例如，假如从 49 个数字中抽取 6 个数，那么程序可能的输出结果为：

```
Bet the following combination. It'll make you rich!
   4
   7
   8
   19
   30
   44
```

要想选择这样一组随机的数字，首先将值 1,2,…,n 填入数组 numbers 中：

```
int[] numbers = new int[n];
for (int i = 0; i < numbers.length; i++)
   numbers[i] = i + 1;
```

第二个数组存放抽取出来的数：

```
int[] result = new int[k];
```

现在可以开始抽取 k 个数了。Math.random 方法将返回一个 0 到 1 之间（包含 0、不包含 1）的随机浮点数。用 n 乘以这个浮点数，可以得到从 0 到 n-1 之间的一个随机数。

```
int r = (int) (Math.random() * n);
```

下面将 result 的第 i 个元素设置为该索引对应的数（numbers[r]），最初是 r+1，但正如所看到的，numbers 数组的内容在每一次抽取之后都会发生变化。

```
result[i] = numbers[r];
```

现在，必须确保不会再次抽到那个数，因为所有抽彩数字必须各不相同。因此，这里用数组中的最后一个数覆盖 number[r]，并将 n 减 1。

```
numbers[r] = numbers[n - 1];
n--;
```

关键在于每次抽取的都是*索引*，而不是实际的值。这个索引指向一个数组，其中包含尚未抽取过的值。

在抽取了 k 个数之后，可以对 result 数组进行排序，来得到更美观的输出：

```
Arrays.sort(result);
for (int r : result)
   System.out.println(r);
```

程序清单 3-7　LotteryDrawing/LotteryDrawing.java

```
1  import java.util.*;
2
3  /**
4   * This program demonstrates array manipulation.
5   * @version 1.20 2004-02-10
6   * @author Cay Horstmann
7   */
8  public class LotteryDrawing
9  {
10    public static void main(String[] args)
11    {
12       Scanner in = new Scanner(System.in);
13
14       System.out.print("How many numbers do you need to draw? ");
15       int k = in.nextInt();
16
17       System.out.print("What is the highest number you can draw? ");
18       int n = in.nextInt();
19
20       // fill an array with numbers 1 2 3 . . . n
21       int[] numbers = new int[n];
22       for (int i = 0; i < numbers.length; i++)
23          numbers[i] = i + 1;
24
25       // draw k numbers and put them into a second array
26       int[] result = new int[k];
27       for (int i = 0; i < result.length; i++)
28       {
29          // make a random index between 0 and n - 1
30          int r = (int) (Math.random() * n);
31
32          // pick the element at the random location
33          result[i] = numbers[r];
34
35          // move the last element into the random location
36          numbers[r] = numbers[n - 1];
37          n--;
```

```
38        }
39
40        // print the sorted array
41        Arrays.sort(result);
42        System.out.println("Bet the following combination. It'll make you rich!");
43        for (int r : result)
44           System.out.println(r);
45     }
46  }
```

API java.util.Arrays 1.2

- `static String toString(xxx[] a)` **5**
  返回一个字符串，其中包含 a 中的元素，这些元素用中括号包围，并用逗号分隔。在这个方法以及后面的方法中，数组元素类型 xxx 可以是 int、long、short、char、byte、boolean、float 或 double。
- `static xxx[] copyOf(xxx[] a, int end)` **6**
- `static xxx[] copyOfRange(xxx[] a, int start, int end)` **6**
  返回与 a 类型相同的一个数组，其长度为 end 或者 end-start，并填入 a 的值。如果 end 大于 a.length，结果会填充 0 或 false 值。
- `static void sort(xxx[] a)`
  使用优化的快速排序算法对数组进行排序。
- `static int binarySearch(xxx[] a, xxx v)`
- `static int binarySearch(xxx[] a, int start, int end, xxx v)` **6**
  使用二分查找算法在有序数组 a 中查找值 v。如果找到 v，则返回相应的索引；否则，返回一个负数值 r。-r-1 是 v 应插入的位置（为保持 a 有序）。
- `static void fill(xxx[] a, xxx v)`
  将数组的所有元素设置为 v。
- `static boolean equals(xxx[] a, xxx[] b)`
  如果两个数组长度相同，并且相同索引对应的元素都相同，则返回 true。

3.10.7 多维数组

多维数组使用多个索引访问数组元素，它适用于表格或其他更复杂的排列形式。你可以先跳过这一节的内容，等真正需要使用这种存储机制时再返回来学习。

假设需要建立一个数值表格，用来显示在不同利率下投资 10 000 美元有多少收益，利息每年兑现并复投（见表 3-8）。

表 3-8 不同利率下的投资收益情况

10%	11%	12%	13%	14%	15%
10 000.00	10 000.00	10 000.00	10 000.00	10 000.00	10 000.00
11 000.00	11 100.00	11 200.00	11 300.00	11 400.00	11 500.00

（续）

10%	11%	12%	13%	14%	15%
12 100.00	12 321.00	12 544.00	12 769.00	12 996.00	13 225.00
13 310.00	13 676.31	14 049.28	14 428.97	14 815.44	15 208.75
14 641 00	15 180.70	15 735.19	16 304.74	16 889.60	17 490.06
16 105.10	16 850.58	17 623.42	18 424 .35	19 254.15	20 113.57
17 715.61	18 704.15	19 738.23	20 819.52	21 949.73	23 130.61
19 487.17	20 761.60	22 106.81	23 526.05	25 022.69	26 600.20
21 435 89	23 045.38	24 759.63	26 584.44	28 525.86	30 590.23
23 579.48	25 580.37	27 730.79	30 040.42	32 519.49	35 178.76

可以使用一个二维数组（也称为矩阵）来存储这些信息，名为 balances。

在 Java 中，声明一个二维数组相当简单。例如：

```
double[][] balances;
```

数组在进行初始化之前是不能使用的。在这里可以如下初始化：

```
balances = new double[NYEARS][NRATES];
```

其他情况下，如果知道数组元素，可以不调用 new，而直接使用一种简写形式对多维数组进行初始化。例如：

```
int[][] magicSquare =
   {
      {16, 3, 2, 13},
      {5, 10, 11, 8},
      {9, 6, 7, 12},
      {4, 15, 14, 1}
   };
```

一旦初始化数组，就可以利用两对中括号访问单个元素，例如，balances[i][j]。

在示例程序中用到了一个存储利率的一维数组 interest 和一个存储账户余额的二维数组 balances（对应每个年度和利率分别有一个余额），使用初始余额来初始化这个数组的第一行：

```
for (int j = 0; j < balances[0].length; j++)
   balances[0][j] = 10000;
```

然后计算其他行，如下所示：

```
for (int i = 1; i < balances.length; i++)
{
   for (int j = 0; j < balances[i].length; j++)
   {
      double oldBalance = balances[i - 1][j];
      double interest = . . .;
      balances[i][j] = oldBalance + interest;
   }
}
```

程序清单 3-8 给出了完整的程序。

注释：“for each”循环语句不会自动循环处理二维数组的所有元素。它会循环处理行，而这些行本身就是一维数组。要想访问二维数组a的所有元素，需要使用两个嵌套的循环，如下所示：

```
for (double[] row : a)
   for (double value : row)
      do something with value
```

提示：要想快速地打印一个二维数组的元素列表，可以调用：

```
System.out.println(Arrays.deepToString(a));
```

输出格式为：

```
[[16, 3, 2, 13], [5, 10, 11, 8], [9, 6, 7, 12], [4, 15, 14, 1]]
```

程序清单3-8 CompoundInterest/CompoundInterest.java

```
1  /**
2   * This program shows how to store tabular data in a 2D array.
3   * @version 1.40 2004-02-10
4   * @author Cay Horstmann
5   */
6  public class CompoundInterest
7  {
8     public static void main(String[] args)
9     {
10       final double STARTRATE = 10;
11       final int NRATES = 6;
12       final int NYEARS = 10;
13
14       // set interest rates to 10 . . . 15%
15       double[] interestRate = new double[NRATES];
16       for (int j = 0; j < interestRate.length; j++)
17          interestRate[j] = (STARTRATE + j) / 100.0;
18
19       double[][] balances = new double[NYEARS][NRATES];
20
21       // set initial balances to 10000
22       for (int j = 0; j < balances[0].length; j++)
23          balances[0][j] = 10000;
24
25       // compute interest for future years
26       for (int i = 1; i < balances.length; i++)
27       {
28          for (int j = 0; j < balances[i].length; j++)
29          {
30             // get last year's balances from previous row
31             double oldBalance = balances[i - 1][j];
32
33             // compute interest
34             double interest = oldBalance * interestRate[j];
35
36             // compute this year's balances
37             balances[i][j] = oldBalance + interest;
38          }
```

```
39        }
40
41        // print one row of interest rates
42        for (int j = 0; j < interestRate.length; j++)
43           System.out.printf("%9.0f%%", 100 * interestRate[j]);
44
45        System.out.println();
46
47        // print balance table
48        for (double[] row : balances)
49        {
50           // print table row
51           for (double b : row)
52              System.out.printf("%10.2f", b);
53
54           System.out.println();
55        }
56     }
57  }
```

3.10.8　不规则数组

到目前为止，我们看到的数组与其他程序设计语言中的数组没有多大区别。但在底层实际存在着一些细微的差异，有时你可以充分利用这一点：Java 实际上没有多维数组，只有一维数组。多维数组被解释为“数组的数组”。

例如，在前面的示例中，balances 数组实际上是一个包含 10 个元素的数组，而每个元素又是一个由 6 个浮点数组成的数组（请参见图 3-15）。

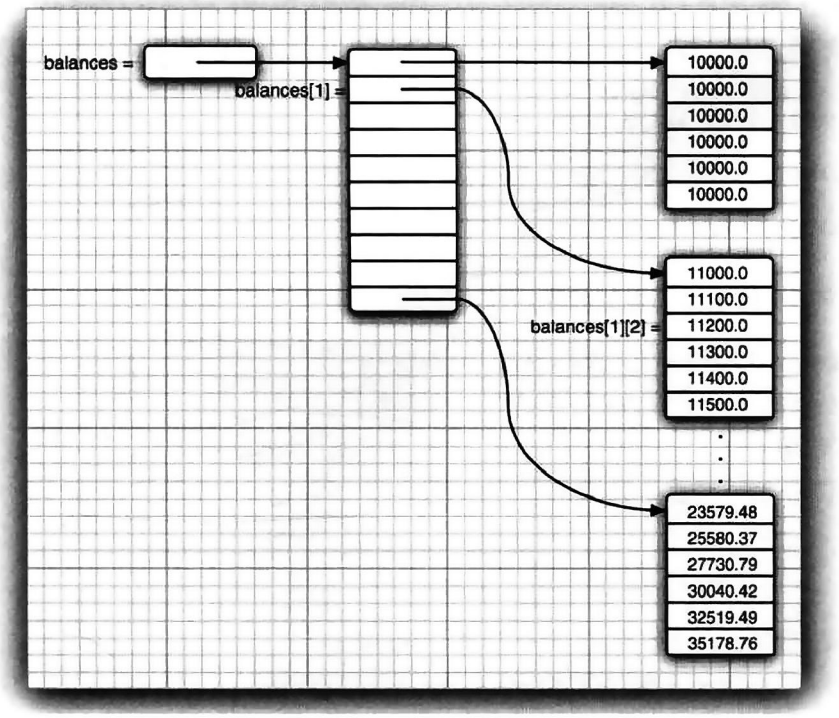


图 3-15　一个二维数组

表达式 balances[i] 指示第 i 个子数组，也就是表格的第 i 行。它本身也是一个数组，balances[i][j] 指示这个数组的第 j 个元素。

由于可以单独地访问数组的某一行，所以可以让两行交换。

```
double[] temp = balances[i];
balances[i] = balances[i + 1];
balances[i + 1] = temp;
```

还可以很容易地构造一个“不规则”数组，即数组的每一行有不同的长度。下面是一个标准的示例。我们将创建一个数组，第 i 行第 j 列的元素将存放“从 i 个数中抽取 j 个数”可能的结果数。

```
1
1  1
1  2  1
1  3  3  1
1  4  6  4  1
1  5 10 10  5 1
1  6 15 20 15 6 1
```

由于 j 不可能大于 i，所以矩阵是三角形的。第 i 行有 i + 1 个元素（允许抽取 0 个元素，这种选择只有一种可能）。要想创建这样一个不规则的数组，首先需要分配一个数组包含这些行：

```
final int NMAX = 10;
int[][] odds = new int[NMAX + 1][];
```

接下来，分配这些行：

```
for (int n = 0; n <= NMAX; n++)
   odds[n] = new int[n + 1];
```

分配了数组之后，可以采用通常的方式访问其中的元素（前提是没有超出边界）：

```
for (int n = 0; n < odds.length; n++)
   for (int k = 0; k < odds[n].length; k++)
   {
      // compute lotteryOdds
      . . .
      odds[n][k] = lotteryOdds;
   }
```

程序清单 3-9 给出了完整的程序。

C++ 注释：在 C++ 中，Java 声明

```
double[][] balances = new double[10][6]; // Java
```

不同于

```
double balances[10][6]; // C++
```

也不同于

```
double (*balances)[6] = new double[10][6]; // C++
```

而是分配了一个包含 10 个指针的数组：

```
double** balances = new double*[10]; // C++
```

然后，这个指针数组的每一个元素会填充一个包含 6 个数字的数组：

```
for (i = 0; i < 10; i++)
   balances[i] = new double[6];
```

庆幸的是，当调用 new double[10][6] 时，这个循环是自动的。需要不规则的数组时，只能单独地分配行数组。

程序清单 3-9　LotteryArray/LotteryArray.java

```
/**
 * This program demonstrates a triangular array.
 * @version 1.20 2004-02-10
 * @author Cay Horstmann
 */
public class LotteryArray
{
   public static void main(String[] args)
   {
      final int NMAX = 10;

      // allocate triangular array
      int[][] odds = new int[NMAX + 1][];
      for (int n = 0; n <= NMAX; n++)
         odds[n] = new int[n + 1];

      // fill triangular array
      for (int n = 0; n < odds.length; n++)
         for (int k = 0; k < odds[n].length; k++)
         {
            /*
             * compute binomial coefficient n*(n-1)*(n-2)*...*(n-k+1)/(1*2*3*...*k)
             */
            int lotteryOdds = 1;
            for (int i = 1; i <= k; i++)
               lotteryOdds = lotteryOdds * (n - i + 1) / i;

            odds[n][k] = lotteryOdds;
         }

      // print triangular array
      for (int[] row : odds)
      {
         for (int odd : row)
            System.out.printf("%4d", odd);
         System.out.println();
      }
   }
}
```

现在，我们已经了解了 Java 语言的基本程序结构，下一章将介绍 Java 面向对象程序设计。

第4章 对象与类

- ▲ 面向对象程序设计概述
- ▲ 使用预定义类
- ▲ 自定义类
- ▲ 静态字段与静态方法
- ▲ 方法参数
- ▲ 对象构造
- ▲ 记录
- ▲ 包
- ▲ JAR 文件
- ▲ 文档注释
- ▲ 类设计技巧

这一章将主要介绍如下内容：

- 面向对象程序设计入门；
- 如何创建标准 Java 类库中类的对象；
- 如何编写自己的类。

如果你没有面向对象程序设计背景，那么一定要认真地阅读本章的内容。面向对象程序设计与面向过程的语言在思维方式上存在着很大的差别。改变思维方式并不是一件很容易的事情，但是为了继续学习 Java，一定要熟悉对象的概念。

对于有经验的 C++ 程序员来说，与上一章一样，对本章的内容不会感到太陌生，但这两种语言还是存在着很多不同之处，所以要认真阅读本章的后半部分内容，你将发现“C++ 注释”对于你转换思维方式会很有帮助。

4.1 面向对象程序设计概述

面向对象程序设计（Object-Oriented Programming，OOP）是当今的主流程序设计范型，它取代了 20 世纪 70 年代的“结构化”或过程式编程技术。由于 Java 是面向对象的，所以你必须熟悉 OOP 才能够很好地使用 Java。

面向对象的程序是由对象组成的，每个对象包含对用户公开的特定功能和隐藏的实现。程序中的很多对象是来自标准类库的“成品”，还有一些是自定义的。究竟是自己构造对象，还是从外界购买，这完全取决于开发项目的预算和时间。但是，从根本上说，只要对象能够满足要求，就不必关心其功能到底是如何实现的。

传统的结构化程序设计通过设计一系列的过程（即*算法*）来求解问题。一旦确定了这些过程，下一步往往要考虑存储数据的适当方式。这就是 Pascal 语言的设计者 Niklaus Wirth 将其著作命名为《算法 + 数据结构 = 程序》（*Algorithms + Data Structures = Programs*, Prentice Hall, 1975）的原因。需要注意的是，在 Wirth 的这个书名中，算法是第一位的，数据结构排

在第二位，这也反映了当时程序员的工作方式。首先，他们会确定操作数据的过程，然后再决定如何组织数据的结构，以便于操作数据。而 OOP 却调换了这个次序，将数据放在第一位，然后再考虑操作数据的算法。

对于一些规模较小的问题，将其分解为过程的做法是合适的，而对象更适合解决规模较大的问题。考虑一个简单的 Web 浏览器，实现这个浏览器可能需要大约 2000 个过程，这些过程需要对一组全局数据进行操作。采用面向对象风格时，可能需要大约 100 个类，每个类平均包含 20 个方法（如图 4-1 所示）。这种结构更易于程序员掌握，也更容易查找 bug。假设一个特定对象的数据出错了，在访问这个数据项的 20 个方法中查找“罪魁祸首”要比在 2000 个过程中查找容易得多。

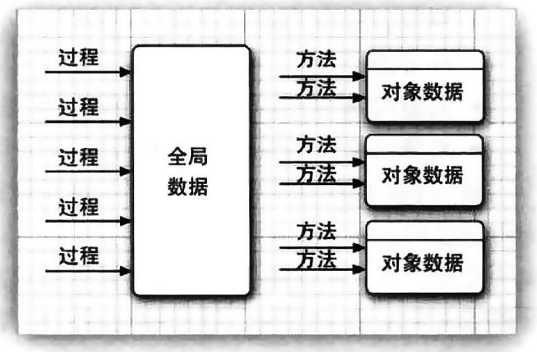


图 4-1 过程式程序设计与面向对象程序设计

4.1.1 类

类（class）指定了如何构造对象。可以将类想象成制作小甜饼的模具，将对象想象为小甜饼。由一个类构造（construct）对象的过程称为创建这个类的一个实例（instance）。

正如前面所看到的，用 Java 编写的所有代码都在某个类中。标准 Java 库提供了几千个类，可用于各种目的，如用户界面设计、日期和日历，以及网络编程。尽管如此，在 Java 中你还需要创建一些自己的类，来描述你的应用相应问题领域中的对象。

封装（encapsulation，有时称为信息隐藏）是处理对象的一个重要概念。从形式上看，封装就是将数据和行为组合在一个包中，并对对象的使用者隐藏具体的实现细节。对象中的数据称为实例字段（instance field），操作数据的过程称为方法（method）。作为一个类的实例，一个特定对象有一组特定的实例字段值。这些值的集合就是这个对象的当前状态（state）。只要在对象上调用一个方法，它的状态就有可能发生改变。

实现封装的关键在于，绝对不能让其他类中的方法直接访问这个类的实例字段。程序只能通过对象的方法与对象数据进行交互。封装为对象赋予了“黑盒”特征，这是提高重用性和可靠性的关键。这意味着一个类可以完全改变存储数据的方式，只要仍旧使用同样的方法操作数据，其他对象就不会知道也不用关心这个类所发生的变化。

OOP 的另一个原则会让用户自定义 Java 类变得更为容易，这就是：可以通过扩展其他类来构建新类。事实上，Java 提供了一个“神通广大的超类”，名为 Object。所有其他类都扩展自这个 Object 类。在下一章中，你会了解更多有关 Object 类的内容。

扩展一个已有的类时，这个新类具有被扩展的那个类的全部属性和方法。你只需要在新类中提供适用于这个新类的新方法和实例字段。通过扩展一个类来得到另外一个类的概念称

为继承（inheritance），有关继承的详细内容请参见下一章。

4.1.2 对象

要想使用 OOP，一定要清楚对象的三个主要特性：

- 对象的行为（behavior）——可以对这个对象做哪些操作，或者可以对这个对象应用哪些方法？
- 对象的状态（state）——调用那些方法时，对象会如何响应？
- 对象的标识（identity）——如何区分可能有相同行为和状态的不同对象？

同一个类的所有实例对象都有一种家族相似性，它们都支持相同的行为。一个对象的行为由所能调用的方法来定义。

此外，每个对象都会保存着描述当前状况的信息，这就是对象的状态。对象的状态可能会随着时间而发生改变，但这种改变不是自发的。对象状态的改变必然是调用方法的结果（如果不经过方法调用就可以改变对象状态，这说明破坏了封装性）。

但是，对象的状态并不能完全描述一个对象，因为每个对象都有一个唯一的标识（identity，或称身份）。例如，在一个订单处理系统中，任何两个订单都是不同的，即使它们订购的商品完全相同。需要注意，作为同一个类的实例，每个对象的标识总是不同的，状态也通常有所不同。

对象的这些关键特性会彼此相互影响。例如，对象的状态会影响它的行为（如果一个订单“已发货”或“已付款”，就应该拒绝要求增删商品的方法调用。反过来，如果订单是“空的”，即还没有订购任何商品，就不应该允许“发货”）。

4.1.3 识别类

传统的过程式程序中，必须从最上面的 main 函数开始编写程序。设计一个面向对象系统时，则没有所谓的“最上面”。因此，学习 OOP 的初学者常常会感觉无从下手。答案是：首先从识别类开始，然后再为各个类添加方法。

识别类的一个简单经验是在分析问题的过程中寻找名词，而方法对应动词。

例如，在订单处理系统中，有这样一些名词：

- 商品（Item）；
- 订单（Order）；
- 发货地址（Shipping address）；
- 付款（Payment）；
- 账户（Account）。

从这些名词就可以得到类 Item、Order 等。

接下来查找动词。商品要添加（add）到订单中，订单可以发货（ship）或取消（cancel），另外可以对订单完成付款（apply）。对于每一个动词，如“添加”“发货”“取消”或者“完成付款”，要识别出负责完成相应动作的对象。例如，当一个新商品添加到订单中时，订单对

象就是负责的对象，因为它知道如何存储商品以及如何对商品进行排序。也就是说，add 应该是 Order 类的一个方法，它接受一个 Item 对象作为参数。

当然，这种“名词与动词”原则只是一种经验，在创建类的时候，只有经验能帮助你确定哪些名词和动词重要。

4.1.4 类之间的关系

类之间最常见的关系有

- 依赖（“uses-a”）；
- 聚合（“has-a”）；
- 继承（“is-a”）。

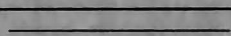
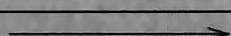
依赖（dependence），即“uses-a”关系，是一种最明显的也最一般的关系。例如，Order 类使用了 Account 类，因为 Order 对象需要访问 Account 对象来查看信用状态。但是 Item 类不依赖于 Account 类，因为 Item 对象不需要考虑客户账户。因此，如果一个类的方法要使用或操作另一个类的对象，我们就说前一个类依赖于后一个类。

应当尽可能减少相互依赖的类。这里的关键是，如果类 A 不知道 B 的存在，它就不会关心 B 的任何改变（这意味着 B 的改变不会在 A 中引入 bug）。用软件工程的术语来说，就是要尽可能减少类之间的耦合（coupling）。

聚合（aggregation），即“has-a”关系，很容易理解，因为这种关系很具体。例如，一个 Order 对象包含一些 Item 对象。包含关系意味着类 A 的对象包含类 B 的对象。

> **注释：**有些方法学家不喜欢聚合这个概念，而更喜欢使用更一般的“关联”关系。从建模的角度看，这是可以理解的。但对于程序员来说，“has-a”关系更加形象。我喜欢使用聚合还有另一个原因：关联的标准记法不是很清楚，请参见表 4-1。

表 4-1 表达类关系的 UML 记法

关 系	UML 连接符
继承	
接口实现	
依赖	
聚合	
关联	
直接关联	

继承（inheritance），即“is-a”关系，表示一个更特殊的类与一个更一般的类之间的关系。例如，RushOrder 类继承了 Order 类。在特殊化的 RushOrder 类中包含一些用于优先处理的特殊方法，还提供了一个计算运费的不同方法；而其他的方法，如添加商品、生成账单等都是从 Order 类继承来的。一般而言，如果类 D 扩展了类 C，类 D 会继承类 C 的方法，另外还会

有一些额外的功能（下一章将详细讨论这个重要的概念）。

很多程序员采用 UML（Unified Modeling Language，统一建模语言）绘制类图，来描述类之间的关系。图 4-2 就是这样一个例子。类用矩形表示，类之间的关系用带有各种修饰的箭头表示。表 4-1 给出了 UML 中最常见的箭头样式。

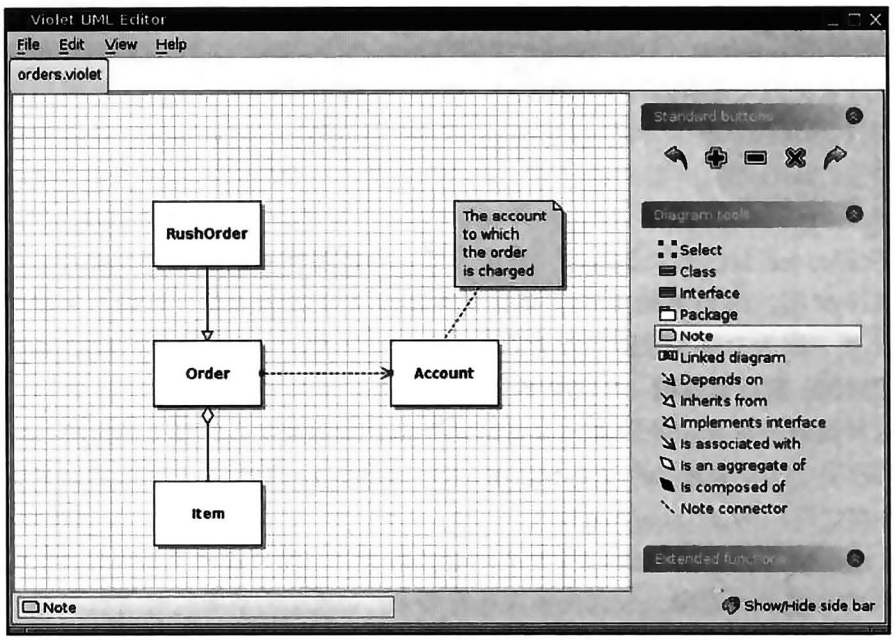


图 4-2 类图

4.2 使用预定义类

在 Java 中，没有类就无法做任何事情，我们前面曾经接触过几个类。然而，并不是所有的类都表现出面向对象的典型特征。以 Math 类为例。你已经看到，可以直接使用 Math 类的方法，如 Math.random，而不必了解它具体是如何实现的，你只需要知道方法名和参数（如果有的话）。这正是封装的关键所在，当然所有类都是这样。但 Math 类只封装了功能，它不需要也不必隐藏数据。由于没有数据，因此也不必考虑创建对象和初始化它们的实例字段，因为根本没有实例字段！

下一节将会介绍一个更典型的类——Date 类，从中可以了解如何构造对象，以及如何调用类的方法。

4.2.1 对象与对象变量

要想使用对象，首先必须构造对象，并指定其初始状态。然后对对象应用方法。

在 Java 程序设计语言中，要使用构造器（constructor，或称构造函数）构造新实例。构造器是一种特殊的方法，其作用是构造并初始化对象。下面来看一个例子。标准 Java 库中包含

一个 Date 类。它的对象可以描述一个时间点，例如，“December 31, 1999, 23:59:59 GMT”。

注释： 你可能会感到奇怪：为什么用类表示日期，而不是像其他语言中那样用一个内置（built-in）类型来表示？例如，Visual Basic 中有一个内置的 date 类型，程序员可以采用 #12/31/1999# 格式指定日期。看起来这似乎很方便，程序员只需要使用内置的 date 类型，而不用考虑类。但实际上，Visual Basic 这样设计合适吗？在有些地区，日期表示为月/日/年，而另外一些地区则表示为日/月/年。语言设计者是否能够预见这些问题呢？如果没有处理好这类问题，语言就有可能陷入混乱，对此感到不满的程序员也会丧失使用这种语言的热情。如果使用类，这些设计任务就交给了类库的设计者。如果类设计得不完善，那么其他程序员可以很容易地编写自己的类，改进或替代（replace）这些系统类（作为印证：Java 的日期类库开始时有些混乱，现在已经重新设计了两次）。

构造器总是与类同名。因此，Date 类的构造器就名为 Date。要想构造一个 Date 对象，需要在构造器前面加上 new 操作符，如下所示：

```
new Date()
```

这个表达式会构造一个新对象。这个对象初始化为当前的日期和时间。

如果需要的话，可以将这个对象传递给一个方法：

```
System.out.println(new Date());
```

或者，可以对刚构造的对象应用一个方法。Date 类中有一个 toString 方法。这个方法将生成日期的一个字符串描述。可以如下对新构造的 Date 对象应用 toString 方法：

```
String s = new Date().toString();
```

在这两个例子中，构造的对象仅使用了一次。通常，你可能希望保留所构造的对象从而能继续使用，为此，需要将对象存放在一个变量中：

```
Date rightNow = new Date();
```

图 4-3 显示了对象变量 rightNow，它引用了新构造的对象。

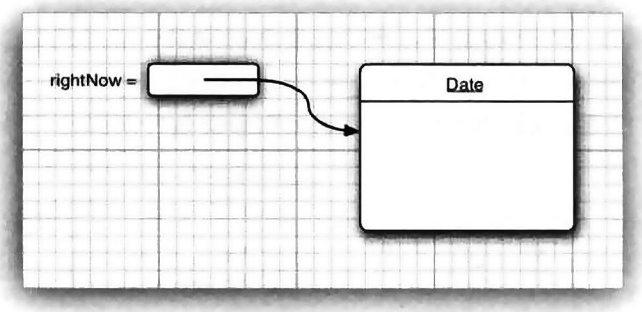


图 4-3 创建一个新对象

对象与对象变量之间存在着一个重要的区别。例如，以下语句

```
Date startTime; // startTime doesn't refer to any object
```

定义了一个对象变量 startTime，它可以引用 Date 类型的对象。但是，一定要认识到：变量 startTime 不是一个对象，而且实际上它甚至还没有引用任何对象。此时不能在这个变量上使用任何 Date 方法。下面的语句

```
s = startTime.toString(); // not yet
```

将产生编译错误。

必须首先初始化 startTime 变量，这里有两个选择。当然，可以初始化这个变量，让它引用一个新构造的对象：

```
startTime = new Date();
```

也可以设置这个变量，让它引用一个已有的对象：

```
startTime = rightNow;
```

现在，这两个变量都引用同一个对象（如图 4-4 所示）。

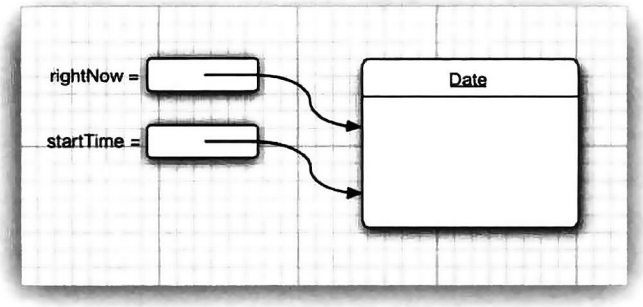


图 4-4　对象变量引用同一个对象

要认识到重要的一点：对象变量并不实际包含一个对象，它只是引用一个对象。

在 Java 中，任何对象变量的值都是一个引用，指向存储在另外一个地方的某个对象。new 操作符的返回值也是一个引用。下面的语句：

```
Date startTime = new Date();
```

有两个部分。表达式 new Date() 构造了一个 Date 类型的对象，它的值是新创建对象的一个引用。再将这个引用存储在 startTime 变量中。

可以显式地将对象变量设置为 null，指示这个对象变量目前没有引用任何对象。

```
startTime = null;
. . .
if (startTime != null)
   System.out.println(startTime);
```

我们将在 4.3.6 节更详细地讨论 null。

C++ 注释：很多人错误地认为 Java 中的对象变量就相当于 C++ 中的引用。然而，C++ 中没有 null 引用，而且引用不能赋值。应当把 Java 中的对象变量看作类似于 C++ 的对象指针。例如，

```
Date rightNow; // Java
```

实际上等同于

```
Date* rightNow; // C++
```

一旦建立了这种关联，一切就清楚了。当然，只有使用了 new 调用后 Date* 指针才会初始化。就这一点而言，C++ 与 Java 的语法几乎是一样的。

```
Date* rightNow = new Date(); // C++
```

如果把一个变量复制到另一个变量，两个变量就指向同一个日期，即它们是同一个对象的指针。Java 中的 null 引用对应于 C++ 中的 NULL 指针。

所有的 Java 对象都存储在堆中。当一个对象包含另一个对象变量时，它只是包含另一个堆对象的指针。

在 C++ 中，指针十分令人头疼，因为它们很容易出错。稍不小心就会创建一个错误的指针，或者使内存管理出问题。在 Java 语言中，这些问题都不复存在。如果使用一个没有初始化的指针，那么运行时系统将会产生一个运行时错误，而不是生成随机的结果。另外，你不必担心内存管理问题，垃圾回收器会处理相关的事宜。

C++ 确实做了很大的努力，它通过支持复制构造器和赋值运算符来实现对象的自动复制。例如，一个链表（linked list）的副本是一个新链表，其内容与原始链表相同，但是有一组独立的链接。这样一来就可以适当地设计类，使它们与内置类型有相同的复制行为。在 Java 中，必须使用 clone 方法获得一个对象的完整副本。

4.2.2 Java 类库中的 LocalDate 类

在前面的例子中，我们使用了 Java 标准类库中的 Date 类。Date 类的实例有一个状态，也就是一个特定的时间点。

尽管在使用 Date 类时不必知道这一点，但时间是用距离一个固定时间点的毫秒数（可正可负）表示的，这个时间点就是所谓的纪元（epoch），它是 UTC 时间 1970 年 1 月 1 日 00:00:00。UTC 就是 Coordinated Universal Time（国际协调时间），与大家熟悉的 GMT（即 Greenwich Mean Time，格林尼治时间）一样，是一种实用的科学标准时间。

但是，Date 类对于处理人类记录日期的日历信息并不是很有用，如“December 31, 1999”。这个特定的日期描述遵循 Gregorian 阳历，这是世界上大多数国家使用的日历。但是，同样的这个时间点采用中国或希伯来的阴历来描述会大不相同，倘若我们有来自火星的顾客，基于他们使用的火星历来描述这个时间点就更不一样了。

注释：有史以来，人类的文明与历法的设计息息相关，日历要为日期指定名字，指定太阳和月亮的周期次序。要了解有关世界上各种日历的有趣解释，从法国大革命的日历到玛雅人计算日期的方法，请参见 Nachum Dershowitz 和 Edward M. Reingold 编写的《Calendrical Calculations》第 4 版（剑桥大学出版社，2018 年）。

类库设计者决定将保存时间与给时间点命名分开。所以，标准 Java 类库分别包含了两个类：一个是用来表示时间点的 Date 类；另一个是用大家熟悉的日历表示法表示日期的 LocalDate 类。Java 8 引入了另外一些类来处理日期和时间的不同方面——有关内容参见卷 II 第 6 章。

将时间度量与日历分开是一种很好的面向对象设计。通常，最好使用不同的类表示不同的概念。

不要使用构造器来构造 LocalDate 类的对象。实际上，应当使用静态工厂方法（factory method），它会代表你调用构造器。下面的表达式：

```
LocalDate.now()
```

会构造一个新对象，表示构造这个对象时的日期。

可以提供年、月和日来构造对应一个特定日期的对象：

```
LocalDate.of(1999, 12, 31)
```

当然，通常我们都希望将构造的对象保存在一个对象变量中：

```
LocalDate newYearsEve = LocalDate.of(1999, 12, 31);
```

一旦有了一个 LocalDate 对象，可以用方法 getYear、getMonthValue 和 getDayOfMonth 得到年、月和日：

```
int year = newYearsEve.getYear(); // 1999
int month = newYearsEve.getMonthValue(); // 12
int day = newYearsEve.getDayOfMonth(); // 31
```

看起来这似乎没有多大的意义，因为这正是构造对象时使用的那些值。不过，有时可能有一个计算得到的日期，然后你希望调用这些方法来了解它的更多信息。例如，plusDays 方法会生成一个新的 LocalDate，如果把应用这个方法的对象称为当前对象，那么这个新日期对象则是距当前对象指定天数的一个新日期：

```
LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);
year = aThousandDaysLater.getYear(); // 2002
month = aThousandDaysLater.getMonthValue(); // 09
day = aThousandDaysLater.getDayOfMonth(); // 26
```

LocalDate 类封装了一些实例字段来维护所设置的日期。如果不查看源代码，就不可能知道类内部的日期表示。当然，封装的意义就在于内部表示并不重要，重要的是类对外提供的方法。

注释： 实际上，Date 类也有得到日、月、年的方法，分别是 getDay、getMonth 以及 getYear，不过这些方法已经**废弃**。当类库设计者意识到某个方法最初就不该引入时，就把它标记为废弃，不鼓励使用。

类库设计者意识到应当单独提供类来处理日历，不过在此之前这些方法已经是 Date 类的一部分了。Java 1.1 中引入较早的一组日历类时，Date 方法被标记为废弃。虽然仍然可以在程序中使用这些方法，不过如果这样做，编译时会出现警告。最好不要使用废弃的方法，因为将来的某个类库版本很有可能会将它们完全删除。

提示：JDK 提供了 jdeprscan 工具来检查你的代码中是否使用了 Java API 已经废弃的特性。有关说明参见 https://docs.oracle.com/en/java/javase/17/docs/specs/man/jdeprscan.html。

4.2.3 更改器方法与访问器方法

再来看上一节中的 plusDays 方法调用：

```
LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);
```

这个调用之后 newYearsEve 会有什么变化？它会改为 1000 天之后的日期吗？事实上，并没有。plusDays 方法会生成一个新的 LocalDate 对象，然后把这个新对象赋给 aThousandDaysLater 变量。原来的对象不做任何改动。我们说 plusDays 方法没有*更改*（mutate）调用这个方法的对象。（这类似于第 3 章中见过的 String 类的 toUpperCase 方法。在一个字符串上调用 toUpperCase 时，这个字符串仍保持不变，并返回一个包含大写字符的新字符串。）

Java 库的一个较早版本曾经有另一个处理日历的类，名为 GregorianCalendar。可以如下为这个类表示的一个日期增加 1000 天：

```
GregorianCalendar someDay = new GregorianCalendar(1999, 11, 31);
   // odd feature of that class: month numbers go from 0 to 11
someDay.add(Calendar.DAY_OF_MONTH, 1000);
```

与 LocalDate.plusDays 方法不同，GregorianCalendar.add 方法是一个*更改器方法*（mutator method）。调用这个方法后，someDay 对象的状态会改变。可以如下查看新状态：

```
year = someDay.get(Calendar.YEAR); // 2002
month = someDay.get(Calendar.MONTH) + 1; // 09
day = someDay.get(Calendar.DAY_OF_MONTH); // 26
```

正是因为这个原因，我们将变量命名为 someDay 而不是 newYearsEve——调用这个更改器方法之后，它不再是新年前夜。

相反，只访问对象而不修改对象的方法有时称为*访问器方法*（accessor method）。例如，LocalDate.getYear 和 GregorianCalendar.get 就是访问器方法。

C++ 注释：在 C++ 中，带有 const 后缀的方法是访问器方法；没有声明为 const 的方法默认为更改器方法。但是，在 Java 语言中，访问器方法与更改器方法在语法上没有明显的区别。

下面用一个具体应用 LocalDate 类的程序来结束这一节。这个程序将显示当前月的日历，格式如下：

```
Mon Tue Wed Thu Fri Sat Sun
                        1
  2   3   4   5   6   7   8
  9  10  11  12  13  14  15
 16  17  18  19  20  21  22
 23  24  25  26* 27  28  29
 30
```

当前日期标记有一个 * 号。可以看到，这个程序需要知道如何计算某月份的天数以及一个给定日期是星期几。

下面来看这个程序的关键步骤。首先构造一个对象，并用当前的日期初始化。

```
LocalDate date = LocalDate.now();
```

下面获得当前的月份和日期。

```
int month = date.getMonthValue();
int today = date.getDayOfMonth();
```

然后，将 date 设置为这个月的第一天，并得到这一天为星期几。

```
date = date.minusDays(today - 1); // set to start of month
DayOfWeek weekday = date.getDayOfWeek();
int value = weekday.getValue(); // 1 = Monday, . . . , 7 = Sunday
```

变量 weekday 设置为 DayOfWeek 类型的对象。我们调用这个对象的 getValue 方法来得到对应星期几的一个数值。这会得到一个整数，这里遵循国际惯例，即周末是一周的结束，星期一就返回 1，星期二返回 2，依此类推。星期日则返回 7。

注意，日历的第一行是缩进的，使当月第一天对应正确的星期几。下面的代码会打印表头和第一行的缩进：

```
System.out.println("Mon Tue Wed Thu Fri Sat Sun");
for (int i = 1; i < value; i++)
   System.out.print("    ");
```

现在我们来打印日历的主体。进入一个循环，其中 date 遍历一个月中的每一天。

每次迭代中，我们要打印日期值。如果 date 是当前日期，这个日期则用一个 * 标记。接下来，把 date 推进到下一天。如果到达新的一周的第一天，则换行打印：

```
while (date.getMonthValue() == month)
{
   System.out.printf("%3d", date.getDayOfMonth());
   if (date.getDayOfMonth() == today)
      System.out.print("*");
   else
      System.out.print(" ");
   date = date.plusDays(1);
   if (date.getDayOfWeek().getValue() == 1) System.out.println();
}
```

什么时候结束呢？我们不知道这个月有几天，是 31 天、30 天、29 天还是 28 天？实际上，只要 date 还在当月就要继续迭代。

程序清单 4-1 给出了完整的程序。

可以看到，利用 LocalDate 类可以编写一个日历程序，它能处理星期几以及各月天数不同等复杂问题。你并不需要知道 LocalDate 类如何计算月和星期几，只需要使用这个类的接口，也就是诸如 plusDays 和 getDayOfWeek 等方法。

这个示例程序的重点是向你展示如何使用一个类的接口来完成相当复杂的任务，而无须了解实现细节。

程序清单 4-1 CalendarTest/CalendarTest.java

```
1  import java.time.*;
2
3  /**
4   * @version 1.5 2015-05-08
5   * @author Cay Horstmann
6   */
7  public class CalendarTest
8  {
9     public static void main(String[] args)
10    {
11       LocalDate date = LocalDate.now();
12       int month = date.getMonthValue();
13       int today = date.getDayOfMonth();
14
15       date = date.minusDays(today - 1); // set to start of month
16       DayOfWeek weekday = date.getDayOfWeek();
17       int value = weekday.getValue(); // 1 = Monday, . . . , 7 = Sunday
18
19       System.out.println("Mon Tue Wed Thu Fri Sat Sun");
20       for (int i = 1; i < value; i++)
21          System.out.print("    ");
22       while (date.getMonthValue() == month)
23       {
24          System.out.printf("%3d", date.getDayOfMonth());
25          if (date.getDayOfMonth() == today)
26             System.out.print("*");
27          else
28             System.out.print(" ");
29          date = date.plusDays(1);
30          if (date.getDayOfWeek().getValue() == 1) System.out.println();
31       }
32       if (date.getDayOfWeek().getValue() != 1) System.out.println();
33    }
34 }
```

API java.time.LocalDate 8

- static LocalDate now()

  构造一个表示当前日期的对象。

- static LocalDate of(int year, int month, int day)

  构造一个表示给定日期的对象。

- int getYear()
- int getMonthValue()
- int getDayOfMonth()

  得到当前日期的年、月和日。

- DayOfWeek getDayOfWeek()

  得到当前日期是星期几，作为 DayOfWeek 类的一个实例返回。在 DayOfWeek 实例上调用

getValue 来得到 1 ～ 7 之间的一个数，表示这是星期几，1 表示星期一，7 表示星期日。

- LocalDate plusDays(int n)
- LocalDate minusDays(int n)

  生成当前日期之后或之前 n 天的日期。

4.3 自定义类

在第 3 章中，我们已经开始编写一些简单的类。但是，那些类都只包含一个简单的 main 方法。现在来学习如何编写更复杂的应用所需要的那种主力类（workhorse class）。通常，这些类没有 main 方法，而有自己的实例字段和实例方法。要想构建一个完整的程序，会结合使用多个类，其中只有一个类有 main 方法。

4.3.1 Employee 类

在 Java 中，最简单的类定义形式为：

```
class ClassName
{
   field1
   field2
   . . .
   constructor1
   constructor2
   . . .
   method1
   method2
   . . .
}
```

下面看一个非常简单的 Employee 类，编写工资管理系统时可能会用到：

```
class Employee
{
   // instance fields
   private String name;
   private double salary;
   private LocalDate hireDay;

   // constructor
   public Employee(String n, double s, int year, int month, int day)
   {
      name = n;
      salary = s;
      hireDay = LocalDate.of(year, month, day);
   }

   // a method
   public String getName()
   {
      return name;
   }
```

```
   // more methods
   . . .
}
```

这里将这个类的实现分成以下几个部分，并分别在稍后的几节中介绍。不过，首先来看程序清单 4-2，这个程序展示了 Employee 类的实际使用。

程序清单 4-2 EmployeeTest/EmployeeTest.java

```
1  import java.time.*;
2
3  /**
4   * This program tests the Employee class.
5   * @version 1.13 2018-04-10
6   * @author Cay Horstmann
7   */
8  public class EmployeeTest
9  {
10    public static void main(String[] args)
11    {
12       // fill the staff array with three Employee objects
13       Employee[] staff = new Employee[3];
14
15       staff[0] = new Employee("Carl Cracker", 75000, 1987, 12, 15);
16       staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
17       staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
18
19       // raise everyone's salary by 5%
20       for (Employee e : staff)
21          e.raiseSalary(5);
22
23       // print out information about all Employee objects
24       for (Employee e : staff)
25          System.out.println("name=" + e.getName() + ",salary=" + e.getSalary() + ",hireDay="
26             + e.getHireDay());
27    }
28 }
29
30 class Employee
31 {
32    private String name;
33    private double salary;
34    private LocalDate hireDay;
35
36    public Employee(String n, double s, int year, int month, int day)
37    {
38       name = n;
39       salary = s;
40       hireDay = LocalDate.of(year, month, day);
41    }
42
43    public String getName()
44    {
45       return name;
46    }
47
```

```
48    public double getSalary()
49    {
50       return salary;
51    }
52
53    public LocalDate getHireDay()
54    {
55       return hireDay;
56    }
57
58    public void raiseSalary(double byPercent)
59    {
60       double raise = salary * byPercent / 100;
61       salary += raise;
62    }
63 }
```

在这个程序中，我们构造了一个 Employee 数组，并填入了 3 个 Employee 对象：

```
Employee[] staff = new Employee[3];

staff[0] = new Employee("Carl Cracker", . . .);
staff[1] = new Employee("Harry Hacker", . . .);
staff[2] = new Employee("Tony Tester", . . .);
```

接下来，使用 Employee 类的 raiseSalary 方法将每个员工的薪水提高 5%：

```
for (Employee e : staff)
   e.raiseSalary(5);
```

最后，调用 getName 方法、getSalary 方法和 getHireDay 方法打印各个员工的信息：

```
for (Employee e : staff)
   System.out.println("name=" + e.getName()
      + ",salary=" + e.getSalary()
      + ",hireDay=" + e.getHireDay());
```

注意，在这个示例程序中包含两个类：Employee 类和带有 public 访问修饰符的 EmployeeTest 类。EmployeeTest 类包含 main 方法，其中使用了前面介绍的代码。

源文件名是 EmployeeTest.java，这是因为文件名必须与 public 类的名字匹配。一个源文件中只能有一个公共类，但可以有任意数目的非公共类。

接下来，编译这段源代码的时候，编译器将在目录中创建两个类文件：EmployeeTest.class 和 Employee.class。

然后启动这个程序，为字节码解释器提供程序中包含 main 方法的那个类的类名：

```
java EmployeeTest
```

字节码解释器开始运行 EmployeeTest 类的 main 方法中的代码。这个代码会先后构造 3 个新 Employee 对象，并显示它们的状态。

4.3.2 使用多个源文件

在程序清单 4-2 中，一个源文件包含了两个类。许多程序员习惯将各个类放在一个单独

的源文件中。例如，将 Employee 类存放在文件 Employee.java 中，而将 EmployeeTest 类存放在文件 EmployeeTest.java 中。

如果喜欢这样组织文件，可以有两种编译源程序的方法。一种是使用通配符调用 Java 编译器：

```
javac Employee*.java
```

这样一来，所有与通配符匹配的源文件都将被编译成类文件。或者可以简单地键入以下命令：

```
javac EmployeeTest.java
```

你可能会感到惊讶，使用第二种方式时并没有显式地编译 Employee.java。不过，当 Java 编译器发现 EmployeeTest.java 中使用了 Employee 类时，它会查找名为 Employee.class 的文件。如果没有找到这个类文件，就会自动搜索 Employee.java 并编译这个文件。另外，如果 Employee.java 的版本较已有的 Employee.class 文件版本更新，Java 编译器就会自动地重新编译这个文件。

注释： 如果熟悉 UNIX 的 make 工具（或者是 Windows 中的相应工具，如 nmake），那么可以认为 Java 编译器内置了 make 功能。

4.3.3 剖析 Employee 类

下面各小节将对 Employee 类进行剖析。首先从这个类的方法开始。通过查看源代码会发现，这个类包含一个构造器和 4 个方法：

```
public Employee(String n, double s, int year, int month, int day)
public String getName()
public double getSalary()
public LocalDate getHireDay()
public void raiseSalary(double byPercent)
```

这个类的所有方法都被标记为 public。关键字 public 意味着任何类的任何方法都可以调用这些方法（共有 4 种访问级别，将在本章和下一章中介绍）。

接下来，需要注意在 Employee 类的实例中有 3 个实例字段，用来存放将要操作的数据：

```
private String name;
private double salary;
private LocalDate hireDay;
```

关键字 private 确保只有 Employee 类本身的方法能够访问这些实例字段，任何其他类的方法都不能读写这些字段。

注释： 可以用 public 标记实例字段，但这是一种很不好的做法。public 实例字段允许程序的任何部分都能对其进行读取和修改，这就完全破坏了封装。任何类的任何方法都可以修改 public 字段，从我们的经验来看，有些代码将利用这种做法存取权限，而这是我们最不希望看到的。因此，这里强烈建议将实例字段标记为 private。

最后，请注意，有两个实例字段本身就是对象：name 字段是 String 类对象的引用，

hireDay 字段是 LocalDate 类对象的引用。类经常包含类类型的实例字段。

4.3.4 从构造器开始

下面先看看 Employee 类的构造器：

```
public Employee(String n, double s, int year, int month, int day)
{
   name = n;
   salary = s;
   hireDay = LocalDate.of(year, month, day);
}
```

可以看到，构造器与类同名。构造 Employee 类的对象时，构造器会运行，这会将实例字段初始化为所希望的初始状态。

例如，使用下面这个代码创建 Employee 类的一个实例时：

```
new Employee("James Bond", 100000, 1950, 1, 1)
```

将如下设置实例字段：

```
name = "James Bond";
salary = 100000;
hireDay = LocalDate.of(1950, 1, 1); // January 1, 1950
```

构造器与其他方法有一个重要的不同。构造器总是结合 new 操作符来调用。不能对一个已经存在的对象调用构造器来重新设置实例字段。例如，

```
james.Employee("James Bond", 250000, 1950, 1, 1) // ERROR
```

将产生编译错误。

本章稍后还会更详细地介绍有关构造器的内容。现在只需要记住：

- 构造器与类同名。
- 每个类可以有一个以上的构造器。
- 构造器可以有 0 个、1 个或多个参数。
- 构造器没有返回值。
- 构造器总是结合 new 操作符一起调用。

C++ 注释： Java 中构造器的工作方式与 C++ 中相同。但是，要记住所有 Java 对象都是在堆中构造的，构造器总是结合 new 操作符一起使用。C++ 程序员最易犯的错误就是忘记 new 操作符：

```
Employee number007("James Bond", 100000, 1950, 1, 1); // C++, not Java
```

这条语句在 C++ 中能够正常运行，但在 Java 中却不行。

警告： 请注意，不要引入与实例字段同名的局部变量。例如，下面的构造器将不会设置 name 或 salary 实例字段：

```
public Employee(String n, double s, . . .)
{
   String name = n; // ERROR
```

```
        double salary = s; // ERROR
        . . .
    }
```

这个构造器声明了**局部**变量 name 和 salary。这些变量只能在构造器内部访问，它们会**遮蔽**（shadow）同名的实例字段。有些程序员偶尔会不假思索地写出这类代码，因为他们的手指会不自觉地增加数据类型。这种错误很难检查出来，因此，必须注意在所有的方法中都不要使用与实例字段同名的变量。

4.3.5 用 var 声明局部变量

在 Java 10 中，如果可以从变量的初始值推导出它们的类型，那么可以用 var 关键字声明局部变量，而无须指定类型。例如，可以不这样声明：

```
Employee harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
```

只需要写为：

```
var harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
```

这一点很好，因为这样可以避免重复写类型名 Employee。

从现在开始，倘若无须了解 Java API 就能从等号右边明显看出类型，在这种情况下我们都将使用 var 表示法。不过我们不会对数值类型使用 var，如 int、long 或 double，这样你就不用当心 0、0L 和 0.0 之间的区别。对 Java API 有了更多经验后，你可能会希望更多地使用 var 关键字。

注意 var 关键字只能用于方法中的局部变量。参数和字段的类型必须声明。

4.3.6 使用 null 引用

在 4.2.1 节中我们已经了解到，对象变量包含一个对象的引用，或者包含一个特殊值 null，后者表示没有引用任何对象。

听上去这是一种处理特殊情况的便捷机制，如未知的名字或雇用日期。不过使用 null 值时要非常小心。

如果对 null 值应用一个方法，会产生一个 NullPointerException 异常。

```
LocalDate rightNow = null;
String s = rightNow.toString(); // NullPointerException
```

这是一个很严重的错误，类似于“索引越界”异常。如果你的程序没有“捕获”异常，那么程序就会终止。正常情况下，程序并不捕获这些异常，而是依赖于程序员从一开始就不要带来异常。

提示：程序因 NullPointerException 异常终止时，栈轨迹会显示问题出现在哪一行代码中。从 Java 17 开始，错误消息会包含有 null 值的变量或方法名。例如，在以下调用中：

```
String s = e.getHireDay().toString();
```

错误消息会告诉你 e 是否为 null 或者 getHireDay 是否返回 null。

定义一个类时，最好清楚地知道哪些字段可能为 null。在我们的例子中，我们不希望 name 或 hireDay 字段为 null。(不用担心 salary 字段，这个字段是基本类型，所以不可能是 null。)

hireDay 字段肯定是非 null 的，因为它初始化为一个新的 LocalDate 对象。但是 name 可能为 null，如果调用构造器时为 n 提供的实参是 null，name 就会是 null。

对此有两种解决方法。“宽容”方法是把 null 参数转换为一个适当的非 null 值：

```
if (n == null) name = "unknown"; else name = n;
```

Objects 类对此提供了一个便利方法：

```
public Employee(String n, double s, int year, int month, int day)
{
   name = Objects.requireNonNullElse(n, "unknown");
   . . .
}
```

“严格”方法则干脆拒绝 null 参数：

```
public Employee(String n, double s, int year, int month, int day)
{
   name = Objects.requireNonNull(n, "The name cannot be null");
   . . .
}
```

如果用 null 名字构造一个 Employee 对象，就会产生 NullPointerException 异常。乍看上去这种补救方法好像不太有用，不过这种方法有两个好处：

1. 异常报告会提供这个问题的描述。

2. 异常报告会准确地指出问题所在的位置，否则 NullPointerException 异常会出现在其他地方，而很难追踪到真正导致问题的构造器参数。

注释：如果要接受一个对象引用作为构造参数，就要问问自己：是不是真的希望接受可有可无的值。如果不是，那么“严格”方法更合适。

4.3.7 隐式参数与显式参数

方法会操作对象并访问它们的实例字段。例如，以下方法

```
public void raiseSalary(double byPercent)
{
   double raise = salary * byPercent / 100;
   salary += raise;
}
```

将调用这个方法的对象的 salary 实例字段设置为一个新值。考虑下面这个调用：

```
number007.raiseSalary(5);
```

其作用是将 number007.salary 字段的值增加 5%。具体地说，这个调用将执行以下指令：

```
double raise = number007.salary * 5 / 100;
number007.salary += raise;
```

raiseSalary 方法有两个参数。第一个参数称为隐式（implicit）参数，是出现在方法名

前的 Employee 类型的对象。第二个参数是位于方法名后面括号中的数值，这是一个显式（explicit）参数。（有人把隐式参数称为方法调用的目标或接收者。）

可以看到，显式参数显式地列在方法声明中，例如 double byPercent。隐式参数则没有出现在方法声明中。

在每一个方法中，关键字 this 指示隐式参数。如果愿意，可以如下改写 raiseSalary 方法：

```
public void raiseSalary(double byPercent)
{
   double raise = this.salary * byPercent / 100;
   this.salary += raise;
}
```

有些程序员更偏爱这样的风格，因为这样可以将实例字段与局部变量明显地区分开来。

C++ 注释：在 C++ 中，通常在类的外面定义方法：

```
void Employee::raiseSalary(double byPercent) // C++, not Java
{
   . . .
}
```

如果在类的内部定义方法，那么这个方法将自动成为内联（inline）方法。

```
class Employee
{
   . . .
   int getName() { return name; } // inline in C++
}
```

在 Java 中，所有的方法都必须在类的内部定义，但这并不表示它们是内联方法。是否将某个方法设置为内联方法是 Java 虚拟机的任务。即时编译器会关注那些简短、经常调用而且没有被覆盖的方法调用，并进行优化。

4.3.8 封装的优点

最后再仔细看一下非常简单的 getName 方法、getSalary 方法和 getHireDay 方法。

```
public String getName()
{
   return name;
}

public double getSalary()
{
   return salary;
}

public LocalDate getHireDay()
{
   return hireDay;
}
```

这些都是典型的访问器方法。由于它们只返回实例字段的值，因此又称为字段访问器

(field accessor)。

如果将 name、salary 和 hireDay 字段标记为公共，而不是编写单独的访问器方法，不是更容易一些吗？

不过，name 是一个只读字段，一旦在构造器中设置，就没有办法能够修改这个字段。这样我们可以确保 name 字段不会受到外界的破坏。

虽然 salary 不是只读字段，但是它只能用 raiseSalary 方法修改。具体地，如果这个值出现了错误，那么只需要调试这个 raiseSalary 方法就可以了。如果 salary 字段是公共的，破坏这个字段值的罪魁祸首有可能出没在任何地方（那就很难调试了）。

有些时候，可能想要获得或设置实例字段的值，那么你需要提供下面三项内容：

- 一个私有的实例字段；
- 一个公共的字段访问器方法；
- 一个公共的字段更改器方法。

这样做要比提供一个简单的公共实例字段复杂些，但有很多明显的好处。

首先，可以改变内部实现，而不影响该类方法之外的任何其他代码。例如，如果将存储姓名的字段改为：

```
String firstName;
String lastName;
```

那么 getName 方法可以改为返回

```
firstName + " " + lastName
```

这个修改对于程序的其余部分是完全不可见的。

当然，为了进行新旧数据表示之间的转换，访问器方法和更改器方法可能需要做许多工作。这将为我们带来第二点好处：更改器方法可以完成错误检查，而只对字段赋值的代码不会费心这么做。例如，setSalary 方法可以检查工资是否小于 0。

警告：注意不要编写返回可变对象引用的访问器方法。在本书之前的一版中，我们的 Employee 类就违反了这个设计原则，其中的 getHireDay 方法返回了一个 Date 类对象：

```
class Employee
{
   private Date hireDay;
   . . .
   public Date getHireDay()
   {
      return hireDay; // BAD
   }
   . . .
}
```

LocalDate 类没有更改器方法，与之不同，Date 类有一个更改器方法 setTime，可以设置毫秒数。

Date 对象是可变的，这一点就破坏了封装性！考虑下面这段有问题的代码：

```
Employee harry = . . .;
Date d = harry.getHireDay();
double tenYearsInMilliseconds = 10 * 365.25 * 24 * 60 * 60 * 1000;
d.setTime(d.getTime() - (long) tenYearsInMilliseconds);
// let's give Harry ten years of added seniority
```

出错的原因很微妙。d 和 harry.hireDay 引用同一个对象（如图 4-5 所示）。对 d 调用更改器方法会自动改变这个 Employee 对象的私有状态！

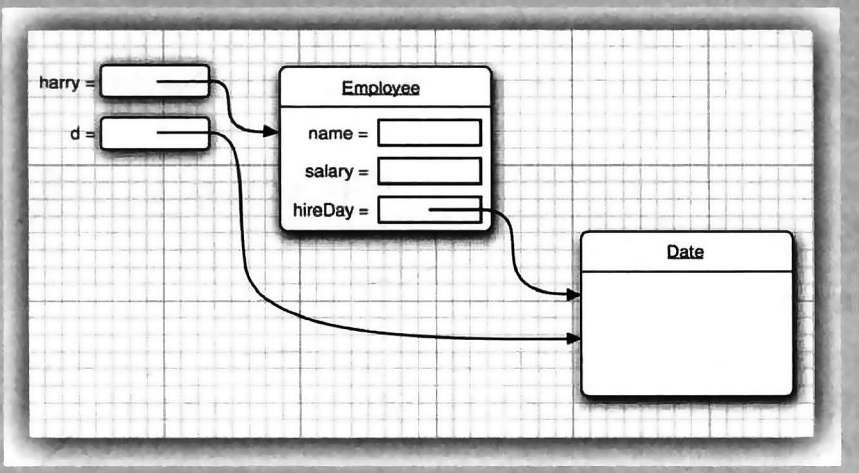


图 4-5 返回可变实例字段的引用

如果需要返回一个可变对象的引用，首先应该对它进行**克隆**（clone）。对象克隆是指存放在另一个新位置上的对象副本。有关对象克隆的详细内容将在第 6 章中讨论。下面是修改后的代码：

```
class Employee
{
   . . .
   public Date getHireDay()
   {
      return (Date) hireDay.clone(); // OK
   }
   . . .
}
```

这里有一个经验，如果需要返回一个可变字段的副本，就应该使用 clone。

4.3.9 基于类的访问权限

从前面已经知道，方法可以访问调用这个方法的对象的私有数据。一个类的方法可以访问这个类的所有对象的私有数据，这令很多人感到奇怪。例如，下面来看用来比较两个员工的 equals 方法。

```
class Employee
```

```
{
   . . .
   public boolean equals(Employee other)
   {
      return name.equals(other.name);
   }
}
```

下面是一个典型的调用：

```
if (harry.equals(boss)) . . .
```

这个方法访问 harry 的私有字段，这并不让人奇怪，不过，它还访问了 boss 的私有字段。这是合法的，其原因是 boss 是 Employee 类型的对象，而 Employee 类的方法可以访问任何 Employee 类型对象的私有字段。

C++ 注释：C++ 也有同样的规则。方法可以访问所属类任何对象的私有特性（feature），而不仅限于隐式参数。

4.3.10　私有方法

实现一个类时，我们会将所有实例字段都设置为私有字段，因为公共数据很危险。不过，方法又应该如何设置呢？尽管大多数方法都是公共的，但在某些情况下，私有方法可能很有用。有时，你可能希望将一个计算代码分解成若干个独立的辅助方法。通常，这些辅助方法不应该成为公共接口的一部分，这是因为它们往往与当前实现关系非常紧密，或者需要一个特殊协议或调用次序。最好将这样的方法实现为私有方法。

在 Java 中，要实现一个私有方法，只需将关键字 public 改为 private 即可。

如果将一个方法设置为私有，倘若你改变了方法的具体实现，并没有义务保证这个方法依然可用。如果数据的表示发生了变化，那么这个方法可能变得更难实现，或者不再需要；这并不重要。重点在于，只要方法是私有的，类的设计者就可以确信它不会在别处使用，所以可以将其删去。如果一个方法是公共的，就不能简单地将其删除，因为可能会有其他代码依赖这个方法。

4.3.11　final 实例字段

可以将实例字段定义为 final。这样的字段必须在构造对象时初始化。也就是说，必须确保在每一个构造器执行之后，这个字段的值已经设置，并且以后不能再修改这个字段。例如，可以将 Employee 类中的 name 字段声明为 final，因为在对象构造之后，这个值不会再改变，即没有 setName 方法。

```
class Employee
{
   private final String name;
   . . .
}
```

final 修饰符对于类型为基本类型或者不可变类的字段尤其有用。（如果类中的所有方法都不会改变其对象，这样的类就是不可变类。例如，String 类就是不可变的。）

对于可变类，使用 final 修饰符可能会造成混乱。例如，考虑以下字段：

```
private final StringBuilder evaluations;
```

它在 Employee 构造器中初始化为

```
evaluations = new StringBuilder();
```

final 关键字只是表示存储在 evaluations 变量中的对象引用不会再指示另一个不同的 StringBuilder 对象。不过这个对象可以更改：

```
public void giveGoldStar()
{
   evaluations.append(LocalDate.now() + ": Gold star!\n");
}
```

4.4 静态字段与静态方法

在前面给出的示例程序中，main 方法都标记了 static 修饰符。下面来讨论这个修饰符的含义。

4.4.1 静态字段

如果将一个字段定义为 static，那么这个字段并不出现在每个类的对象中。每个静态字段只有一个副本。可以认为静态字段属于类，而不属于单个对象。例如，假设需要为每一个员工分配唯一的标识码，这里为 Employee 类添加一个实例字段 id 和一个静态字段 nextId：

```
class Employee
{
   private static int nextId = 1;

   private int id;
   . . .
}
```

现在，每一个 Employee 对象都有自己的 id 字段，但这个类的所有实例将共享一个 nextId 字段。换句话说，如果有 1000 个 Employee 类对象，则有 1000 个实例字段 id，每个对象有一个实例字段 id。但是，只有一个静态字段 nextId。即使没有 Employee 对象，静态字段 nextId 也存在。它属于类，而不属于任何单个对象。

注释：在一些面向对象程序设计语言中，静态字段被称为类字段。术语“静态”只是沿用了 C++ 的叫法，并无实际意义。

在构造器中，我们为新 Employee 对象分配下一个可用的 ID，然后将其增 1：

```
id = nextId;
nextId++;
```

假设我们构造了对象 harry。harry 的 id 字段设置为静态字段 nextId 的当前值，并将静态字段 nextId 的值加 1：

```
harry.id = Employee.nextId;
Employee.nextId++;
```

4.4.2 静态常量

静态变量使用得比较少，但静态常量却很常用。例如，Math 类中定义了一个静态常量：

```
public class Math
{
   . . .
   public static final double PI = 3.14159265358979323846;
   . . .
}
```

在你的程序中，可以用 Math.PI 来访问这个常量。

如果省略关键字 static，那么 PI 就变成了 Math 类的一个实例字段。也就是说，需要通过 Math 类的一个对象来访问 PI，并且每一个 Math 对象都有它自己的一个 PI 副本。

另一个你已经多次使用的静态常量是 System.out。它在 System 类中声明如下：

```
public class System
{
   . . .
   public static final PrintStream out = . . .;
   . . .
}
```

前面曾经多次提到过，最好不要有公共字段，因为谁都可以修改公共字段。不过，公共常量（即 final 字段）却没问题。因为 out 被声明为 final，所以，不允许再将它重新赋值为另一个打印流：

```
System.out = new PrintStream(. . .); // ERROR--out is final
```

注释：如果查看 System 类，就会发现有一个 setOut 方法可以将 System.out 设置为不同的流。你可能会感到奇怪，为什么这个方法可以修改 final 变量的值。原因在于，setOut 方法是一个原生方法，而不是在 Java 语言中实现的。原生方法可以绕过 Java 语言的访问控制机制。这是一种特殊的解决方法，你自己编写程序时不要模仿这种做法。

4.4.3 静态方法

静态方法是不操作对象的方法。例如，Math 类的 pow 方法就是一个静态方法。以下表达式：

```
Math.pow(x, a)
```

会计算幂 x^a。它并不使用任何 Math 对象来完成这个任务。换句话说，它没有隐式参数。

可以认为静态方法是没有 this 参数的方法（在一个非静态方法中，this 参数指示这个方法的隐式参数，参见 4.3.7 节）。

Employee 类的静态方法不能访问 id 实例字段，因为它并不操作对象。但是，静态方法可以访问静态字段。下面是这样一个静态方法的示例：

```
public static int advanceId()
{
   int r = nextId; // obtain next available id
   nextId++;
   return r;
}
```

要调用这个方法，需要提供类名：

```
int n = Employee.advanceId();
```

这个方法可以省略关键字 static 吗？答案是肯定的。但是，这样一来，你就需要通过 Employee 类型的对象引用来调用这个方法。

注释： 可以使用对象调用静态方法，这是合法的。例如，如果 harry 是一个 Employee 对象，那么可以调用 harry.advanceId() 而不是 Employee.advanceId()。不过，我发现这种写法很容易造成混淆，其原因是 advanceId 方法计算的结果与 harry 毫无关系。我们建议使用类名而不是对象来调用静态方法。

下面两种情况可以使用静态方法：

- 方法不需要访问对象状态，因为它需要的所有参数都通过显式参数提供（例如 Math.pow）。
- 方法只需要访问类的静态字段（例如 Employee. advanceId）。

C++ 注释： Java 中的静态字段与静态方法在功能上与 C++ 相同。但是，语法稍有所不同。在 C++ 中，要使用 :: 操作符访问作用域之外的静态字段或静态方法，如 Math::PI。

术语“静态”有一段不寻常的历史。起初，C 引入关键字 static 是为了表示退出一个块后依然存在的局部变量。在这种情况下，术语“静态”是有意义的：变量一直保留，当再次进入这个块时它仍然存在。随后，static 在 C 中有了第二种含义，表示不能从其他文件访问的全局变量和函数。重用关键字 static 只是为了避免引入一个新的关键字。最后，C++ 第三次重用了这个关键字，与之前赋予的含义完全无关，它指示属于类而不属于任何特定类对象的变量和函数。这与 Java 中这个关键字的含义相同。

4.4.4 工厂方法

静态方法还有另外一种常见的用途。类似 LocalDate 和 NumberFormat 的类使用静态工厂方法（factory method）来构造对象。你已经见过工厂方法 LocalDate.now 和 LocalDate.of。可以如下得到不同样式的格式化对象：

```
NumberFormat currencyFormatter = NumberFormat.getCurrencyInstance();
NumberFormat percentFormatter = NumberFormat.getPercentInstance();
double x = 0.1;
System.out.println(currencyFormatter.format(x)); // prints $0.10
System.out.println(percentFormatter.format(x)); // prints 10%
```

为什么 NumberFormat 类不使用构造器来创建对象呢？这有两个原因：

- 无法为构造器命名。构造器的名字总是要与类名相同。但是，这里希望有两个不同的名字，分别得到货币实例和百分比实例。

- 使用构造器时，无法改变所构造对象的类型。而工厂方法实际上将返回 DecimalFormat 类的对象，这是继承 NumberFormat 的一个子类（有关继承的更多详细内容请参见第 5 章）。

4.4.5 main 方法

需要指出，可以调用静态方而不需要任何对象。例如，不需要构造 Math 类的任何对象就可以调用 Math.pow。

同理，main 方法也是一个静态方法。

```
public class Application
{
   public static void main(String[] args)
   {
      // construct objects here
      . . .
   }
}
```

main 方法不对任何对象进行操作。事实上，启动程序时还没有任何对象。将执行静态 main 方法，并构造程序所需要的对象。

提示：每一个类都可以有一个 main 方法。这是为类增加演示代码的一个技巧。例如，可以在 Employee 类中添加一个 main 方法：

```
class Employee
{
   public Employee(String n, double s, int year, int month, int day)
   {
      name = n;
      salary = s;
      hireDay = LocalDate.of(year, month, day);
   }
   . . .
   public static void main(String[] args) // unit test
   {
      var e = new Employee("Romeo", 50000, 2003, 3, 31);
      e.raiseSalary(10);
      System.out.println(e.getName() + " " + e.getSalary());
   }
   . . .
}
```

要看 Employee 类的演示，只需要执行

```
java Employee
```

如果 Employee 类是一个更大的应用的一部分，那么可以使用以下命令运行这个应用：

```
java Application
```

Employee 类的 main 方法永远不会执行。

程序清单 4-3 中的程序包含了 Employee 类的一个简单版本，其中有一个静态字段 nextId 和一个静态方法 advanceId。这里将三个 Employee 对象填入一个数组，然后打印员工信息。最后，

打印下一个可用的员工标识码来展示静态方法。

程序清单 4-3 StaticTest/StaticTest.java

```
1  /**
2   * This program demonstrates static methods.
3   * @version 1.02 2008-04-10
4   * @author Cay Horstmann
5   */
6  public class StaticTest
7  {
8     public static void main(String[] args)
9     {
10       // fill the staff array with three Employee objects
11       var staff = new Employee[3];
12
13       staff[0] = new Employee("Tom", 40000);
14       staff[1] = new Employee("Dick", 60000);
15       staff[2] = new Employee("Harry", 65000);
16
17       // print out information about all Employee objects
18       for (Employee e : staff)
19       {
20          System.out.println("name=" + e.getName() + ",id=" + e.getId() + ",salary="
21             + e.getSalary());
22       }
23
24       int n = Employee.getNextId(); // calls static method
25       System.out.println("Next available id=" + n);
26    }
27 }
28
29 class Employee
30 {
31    private static int nextId = 1;
32
33    private String name;
34    private double salary;
35    private int id;
36
37    public Employee(String n, double s)
38    {
39       name = n;
40       salary = s;
41       id = advanceId();
42    }
43
44    public String getName()
45    {
46       return name;
47    }
48
49    public double getSalary()
50    {
51       return salary;
```

```
52    }
53
54    public int getId()
55    {
56       return id;
57    }
58
59    public static int advanceId()
60    {
61       int r = nextId; // obtain next available id
62       nextId++;
63       return r;
64    }
65
66    public static void main(String[] args) // unit test
67    {
68       var e = new Employee("Harry", 50000);
69       System.out.println(e.getName() + " " + e.getSalary());
70    }
71 }
```

需要注意，Employee 类也有一个静态 main 方法用于单元测试。试着运行

```
java Employee
```

和

```
java StaticTest
```

执行两个 main 方法。

API java.util.Objects 7

- static <T> void requireNonNull(T obj)
- static <T> void requireNonNull(T obj, String message)
- static <T> void requireNonNull(T obj, Supplier<String> messageSupplier) **8**

  如果 obj 为 null，这些方法会抛出一个 NullPointerException 异常而没有任何消息，或者有给定的消息。(第 6 章会解释如何利用供应者以懒方式得到一个值。第 8 章会解释 <T> 语法。)
- static <T> T requireNonNullElse(T obj, T defaultObj) **9**
- static <T> T requireNonNullElseGet(T obj, Supplier<T> defaultSupplier) **9**

  如果 obj 不为 null 则返回 obj，或者如果 obj 为 null 则返回默认对象。

4.5 方法参数

首先来回顾在程序设计语言中关于如何将参数传递到方法（或函数）的一些专业术语。*按值调用*（call by value）表示方法接收的是调用者提供的值。而*按引用调用*（call by reference）表示方法接收的是调用者提供的变量*位置*（location）。所以，方法可以修改按引用传递的变量的值，而不能修改按值传递的变量的值。“按……调用”（call by）是一个标准的计算机科学

术语，用来描述各种程序设计语言（不只是 Java）中方法参数的行为（事实上，以前还有一种按名调用（call by name），Algol 程序设计语言是最古老的高级语言之一，它就采用了按名调用方式。不过，这种传递方式已经成为历史）。

Java 程序设计语言总是采用按值调用。也就是说，方法会得到所有参数值的一个副本。具体来讲，方法不能修改传递给它的任何参数变量的内容。

例如，考虑下面的调用：

```
double percent = 10;
harry.raiseSalary(percent);
```

不论这个方法具体如何实现，我们知道，在这个方法调用之后，percent 的值还是 10。

下面再仔细研究一下这种情况。假定一个方法试图将一个参数值增加至 3 倍：

```
public static void tripleValue(double x) // doesn't work
{
   x = 3 * x;
}
```

然后调用这个方法：

```
double percent = 10;
tripleValue(percent);
```

不过，这并不起作用。调用这个方法之后，percent 的值还是 10。下面来看发生了什么：

1. x 初始化为 percent 值的一个副本（也就是 10）。

2. x 乘以 3 后等于 30，但是 percent 仍然是 10（如图 4-6 所示）。

3. 这个方法结束之后，参数变量 x 不再使用。

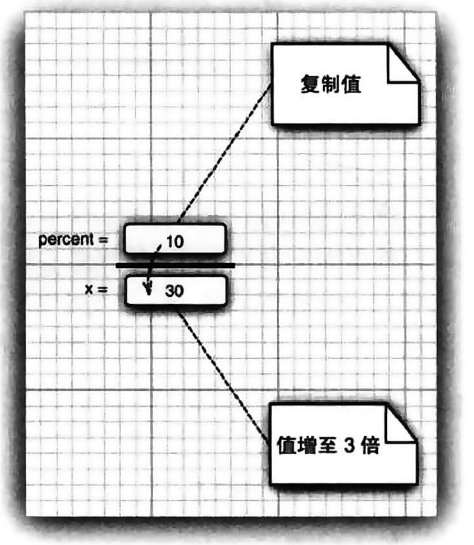


图 4-6 修改参数变量没有持续效果

不过，有两种不同类型的方法参数：

- 基本数据类型（数字、布尔值）。
- 对象引用。

你已经看到，一个方法不可能修改基本数据类型的参数，而对象参数就不同了，可以很容易地实现一个方法将一个员工的工资增至 3 倍：

```
public static void tripleSalary(Employee x) // works
{
   x.raiseSalary(200);
}
```

如下调用这个方法时，

```
harry = new Employee(. . .);
tripleSalary(harry);
```

具体的执行过程为：

1. x 初始化为 harry 值的一个副本，这里就是一个对象引用。

2. raiseSalary 方法应用于这个对象引用。x 和 harry 同时引用的那个 Employee 对象的工资提高了 200%。

3. 方法结束后，参数变量 x 不再使用。当然，对象变量 harry 继续引用那个工资增至 3 倍的员工对象（如图 4-7 所示）。

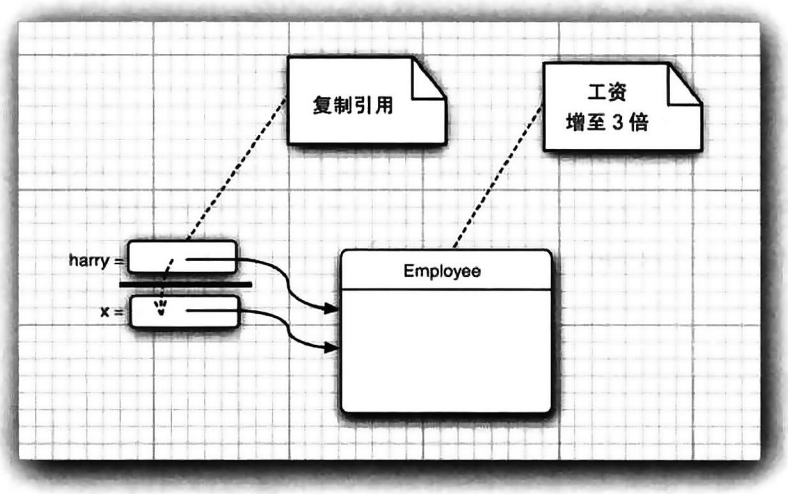


图 4-7　修改参数引用的对象有持续效果

可以看到，实现方法改变对象参数的状态是完全可以的，实际上也相当常见。理由很简单，方法得到的是对象引用的副本，原来的对象引用和这个副本都引用同一个对象。

很多程序设计语言（特别是 C++ 和 Pascal）提供了两种参数传递方式：按值调用和按引用调用。有些程序员（甚至有些书的作者）声称 Java 对对象采用的是按引用调用，实际上，这是不对的。由于这种误解很常见，所以很有必要给出一个反例来详细地说明这个问题。

下面来编写一个交换两个 Employee 对象的方法：

```
public static void swap(Employee x, Employee y) // doesn't work
{
   Employee temp = x;
   x = y;
   y = temp;
}
```

如果 Java 对对象采用的是按引用调用，那么这个方法就应该能够实现交换：

```
var a = new Employee("Alice", . . .);
var b = new Employee("Bob", . . .);
swap(a, b);
// does a now refer to Bob, b to Alice?
```

但是，这个方法并没有改变存储在变量 a 和 b 中的对象引用。swap 方法的参数 x 和 y 初始化为两个对象引用的副本，这个方法交换的是这两个副本。

```
// x refers to Alice, y to Bob
Employee temp = x;
x = y;
```

```
y = temp;
// now x refers to Bob, y to Alice
```

最终，白费力气。方法结束时，参数变量 x 和 y 被丢弃了。原来的变量 a 和 b 仍然引用这个方法调用之前所引用的对象（如图 4-8 所示）。

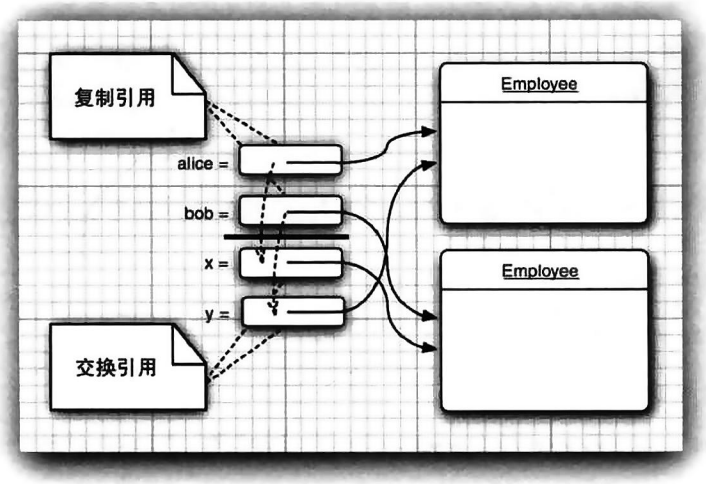


图 4-8 交换参数变量没有持续效果

这说明：Java 程序设计语言对对象采用的不是按引用调用。实际上，对象引用（object reference）是按值传递的。

下面来总结在 Java 中对方法参数能做什么和不能做什么：

- 方法不能修改基本数据类型的参数（即数值型或布尔型）。
- 方法可以改变对象参数的状态。
- 方法不能让一个对象参数引用一个新对象。

程序清单 4-4 中的程序展示了这几点。在这个程序中，首先试图将一个数值参数的值增至 3 倍，但没有成功：

```
Testing tripleValue:
Before: percent=10.0
End of method: x=30.0
After: percent=10.0
```

随后，成功地将一个员工的工资增至 3 倍：

```
Testing tripleSalary:
Before: salary=50000.0
End of method: salary=150000.0
After: salary=150000.0
```

方法结束之后，harry 引用的对象的状态发生了改变。这是因为这个方法可以通过对象引用的副本修改所引用对象的状态。

最后，程序演示了 swap 方法的失败结果：

```
Testing swap:
Before: a=Alice
```

```
Before: b=Bob
End of method: x=Bob
End of method: y=Alice
After: a=Alice
After: b=Bob
```

可以看出，参数变量 x 和 y 交换了，但是变量 a 和 b 没有受到影响。

C++ 注释： C++ 中有按值调用和按引用调用。引用参数标有 & 符号。例如，可以轻松地实现 void tripleValue(double& x) 方法或 void swap(Employee& x, Employee& y) 方法来修改它们的引用参数。

程序清单 4-4 ParamTest/ParamTest.java

```
1  /**
2   * This program demonstrates parameter passing in Java.
3   * @version 1.01 2018-04-10
4   * @author Cay Horstmann
5   */
6  public class ParamTest
7  {
8     public static void main(String[] args)
9     {
10       /*
11        * Test 1: Methods can't modify numeric parameters
12        */
13       System.out.println("Testing tripleValue:");
14       double percent = 10;
15       System.out.println("Before: percent=" + percent);
16       tripleValue(percent);
17       System.out.println("After: percent=" + percent);
18
19       /*
20        * Test 2: Methods can change the state of object parameters
21        */
22       System.out.println("\nTesting tripleSalary:");
23       var harry = new Employee("Harry", 50000);
24       System.out.println("Before: salary=" + harry.getSalary());
25       tripleSalary(harry);
26       System.out.println("After: salary=" + harry.getSalary());
27
28       /*
29        * Test 3: Methods can't attach new objects to object parameters
30        */
31       System.out.println("\nTesting swap:");
32       var a = new Employee("Alice", 70000);
33       var b = new Employee("Bob", 60000);
34       System.out.println("Before: a=" + a.getName());
35       System.out.println("Before: b=" + b.getName());
36       swap(a, b);
37       System.out.println("After: a=" + a.getName());
38       System.out.println("After: b=" + b.getName());
39    }
40
41    public static void tripleValue(double x) // doesn't work
```

```
42    {
43       x = 3 * x;
44       System.out.println("End of method: x=" + x);
45    }
46
47    public static void tripleSalary(Employee x) // works
48    {
49       x.raiseSalary(200);
50       System.out.println("End of method: salary=" + x.getSalary());
51    }
52
53    public static void swap(Employee x, Employee y)
54    {
55       Employee temp = x;
56       x = y;
57       y = temp;
58       System.out.println("End of method: x=" + x.getName());
59       System.out.println("End of method: y=" + y.getName());
60    }
61 }
62
63 class Employee // simplified Employee class
64 {
65    private String name;
66    private double salary;
67
68    public Employee(String n, double s)
69    {
70       name = n;
71       salary = s;
72    }
73
74    public String getName()
75    {
76       return name;
77    }
78
79    public double getSalary()
80    {
81       return salary;
82    }
83
84    public void raiseSalary(double byPercent)
85    {
86       double raise = salary * byPercent / 100;
87       salary += raise;
88    }
89 }
```

4.6 对象构造

前面已经学习了如何编写简单的构造器来定义对象的初始状态。不过，由于对象构造非

常重要，所以 Java 提供了多种编写构造器的机制。下面几节将详细介绍这些机制。

4.6.1 重载

有些类有多个构造器。例如，可以如下构造一个空的 StringBuilder 对象：

```
var messages = new StringBuilder();
```

或者，可以指定一个初始字符串：

```
var todoList = new StringBuilder("To do:\n");
```

这种功能叫作重载（overloading）。如果多个方法（比如，StringBuilder 构造器方法）有相同的方法名但有不同的参数，便出现了重载。编译器必须挑选出具体调用哪个方法。它用各个方法首部中的参数类型与特定方法调用中所使用的值类型进行匹配，来选出正确的方法。如果编译器无法匹配参数，就会产生编译时错误，这可能因为根本不存在匹配，或者所有重载方法中没有一个相对更好的方法（这个查找匹配的过程称为重载解析（overloading resolution））。

注释：Java 允许重载任何方法，而不只是构造器方法。因此，要完整地描述一个方法，需要指定方法名以及参数类型，这叫作方法的签名（signature）。例如，String 类有 4 个名为 indexOf 的公共方法。它们的签名是

```
indexOf(int)
indexOf(int, int)
indexOf(String)
indexOf(String, int)
```

返回类型不是方法签名的一部分。也就是说，不能有两个名字相同、参数类型也相同却有不同返回类型的方法。

4.6.2 默认字段初始化

如果在构造器中没有显式地为一个字段设置初始值，就会将它自动设置为默认值：数值将设置为 0，布尔值为 false，对象引用为 null。有些人认为依赖默认值的做法是一种不好的编程实践。确实，如果不明确地对字段进行初始化，就会影响程序代码的可读性。

注释：这是字段与局部变量的一个重要区别。方法中的局部变量必须明确地初始化。但是在类中，如果没有初始化类中的字段，将会自动初始化为默认值（0、false 或 null）。

例如，考虑 Employee 类。假定没有在构造器中指定如何初始化某些字段，默认情况下，就会将 salary 字段初始化为 0，将 name 和 hireDay 字段初始化为 null。

但是，这并不是一个好主意。如果有人调用 getName 方法或 getHireDay 方法，就会得到一个 null 引用，这可能不是他们想要的结果：

```
LocalDate h = harry.getHireDay();
int year = h.getYear(); // throws exception if h is null
```

4.6.3 无参数的构造器

很多类都包含无参数的构造器，由无参数构造器创建对象时，对象的状态会设置为适当

的默认值。例如，以下是 Employee 类的一个无参数构造器：

```
public Employee()
{
   name = "";
   salary = 0;
   hireDay = LocalDate.now();
}
```

如果你写的类没有构造器，就会为你提供一个无参数构造器。这个构造器将所有的实例字段设置为相应的默认值。所以，实例字段中的所有数值型数据会设置为 0，所有布尔值设置为 false，所有对象变量将设置为 null。

如果类中提供了至少一个构造器，但是没有提供无参数构造器，那么构造对象时就必须提供参数，否则就是不合法的。例如，程序清单 4-2 中的 Employee 类提供了一个构造器：

```
public Employee(String n, double s, int year, int month, int day)
```

对于这个类，构造默认的员工就是不合法的。也就是说，以下调用

```
e = new Employee();
```

将产生错误。

警告： 请记住，仅当类没有任何其他构造器的时候，你才会得到一个默认的无参数构造器。编写类的时候，如果写了一个你自己的构造器，要想让这个类的使用者能够通过以下调用创建一个实例：

```
new ClassName()
```

你就必须提供一个无参数的构造器。当然，如果接受所有字段设置为默认值，则只需要提供以下代码：

```
public ClassName()
{
}
```

C++ 注释： C++ 对于构造字段有一种特殊的初始化器列表语法，如下所示：

```
Employee::Employee(String n, double s, int y, int m, int d) // C++
:  name(n),
   salary(s),
   hireDay(y, m, d)
{
}
```

C++ 使用这种特殊语法来避免不必要地调用无参数构造器。在 Java 中，不需要这种语法，因为对象没有子对象，只有其他对象的引用。

4.6.4 显式字段初始化

通过重载类的构造器方法，可以采用多种形式设置类实例字段的初始状态。不论调用哪个构造器，每个实例字段都要设置为一个有意义的初始值，确保这一点总是一个好主意。

可以在类定义中直接为任何字段赋值。例如：

```
class Employee
{
   private String name = "";
   . . .
}
```

在执行构造器之前完成这个赋值。如果一个类的所有构造器都需要把某个特定的实例字段设置为同一个值，那么这个语法尤其有用。

初始值不一定是常量值。在下面的例子中，就是利用方法调用初始化一个字段。考虑以下 Employee 类，其中每个员工有一个 id 字段。可以使用以下方式进行初始化：

```
class Employee
{
   private static int nextId;
   private int id = advanceId();
   . . .
   private static int advanceId()
   {
      int r = nextId;
      nextId++;
      return r;
   }
   . . .
}
```

4.6.5 参数名

在编写很小的构造器时（这十分常见），在为参数命名时可能有些困惑。

我们通常喜欢用单个字母作为参数名：

```
public Employee(String n, double s)
{
   name = n;
   salary = s;
}
```

但这样做有一个缺点：只有阅读代码才能够了解参数 n 和参数 s 的含义。

有些程序员在每个参数前面加上一个前缀“a”：

```
public Employee(String aName, double aSalary)
{
   name = aName;
   salary = aSalary;
}
```

这样更好一些。读者一眼就能够看懂参数的含义。

还一种常用的技巧，它基于这样的事实：参数变量会遮蔽同名的实例字段。例如，如果将参数命名为 salary，那么 salary 将指示这个参数，而不是实例字段。但是，还是可以用 this.salary 访问实例字段。回想一下，this 指示隐式参数，也就是所构造的对象。下面来看一个示例：

```
public Employee(String name, double salary)
{
   this.name = name;
   this.salary = salary;
}
```

C++ 注释： 在 C++ 中，经常用下画线或某个固定的字母（一般选用 m 或 x）作为实例字段的前缀。例如，salary 字段可能被命名为 _salary、mSalary 或 xSalary。Java 程序员通常不这样做。

4.6.6 调用另一个构造器

关键字 this 指示一个方法的隐式参数。不过，这个关键字还有另外一个含义。

如果构造器的第一个语句形如 this(...)，这个构造器将调用同一个类的另一个构造器。下面是一个典型的例子：

```
public Employee(double s)
{
   // calls Employee(String, double)
   this("Employee #" + nextId, s);
   nextId++;
}
```

当调用 new Employee(60000) 时，Employee(double) 构造器将调用 Employee(String, double) 构造器。

采用这种方式使用 this 关键字非常有用，这样只需要写一次公共构造代码。

C++ 注释： 在 Java 中，this 引用等价于 C++ 中的 this 指针。但是，在 C++ 中，一个构造器不能调用另一个构造器。如果在 C++ 中想抽取出公共的初始化代码，则必须编写一个单独的方法。

4.6.7 初始化块

前面已经介绍过两种初始化实例字段的方法：

- 在构造器中设置值；
- 在声明中赋值。

实际上，Java 还有第三种机制，称为*初始化块*（initialization block）。在一个类的声明中，可以包含任意的代码块。构造这个类的对象时，这些块就会执行。例如，

```
class Employee
{
   private static int nextId;

   private int id;
   private String name;
   private double salary;

   // object initialization block
```

```
    {
       id = nextId;
       nextId++;
    }

    public Employee(String n, double s)
    {
       name = n;
       salary = s;
    }

    public Employee()
    {
       name = "";
       salary = 0;
    }
    . . .
}
```

在这个示例中，无论使用哪个构造器构造对象，id字段都会在对象初始化块中初始化。首先运行初始化块，然后才运行构造器的主体部分。

这种机制不是必需的，也不常见。通常会直接将初始化代码放在构造器中。

注释： 可以在初始化块中设置字段，即使这些字段在类后面才定义，这是合法的。但是，为了避免循环定义，不允许读取在后面初始化的字段。具体规则请参见Java语言规范的8.3.3节（http://docs.oracle.com/javase/specs）。这些规则太过复杂，让编译器的实现者都很头疼，所以较早的Java版本中这些规则的实现存在一些小错误。因此，建议总是将初始化块放在字段定义之后。

由于初始化实例字段有多种途径，所以列出构造过程的所有路径可能让人很费解。下面是调用构造器时的具体处理步骤：

1. 如果构造器的第一行调用了另一个构造器，则基于所提供的参数执行第二个构造器。

2. 否则，

a）所有实例字段初始化为其默认值（0、false或null）。

b）按照在类声明中出现的顺序，执行所有字段初始化方法和初始化块。

3. 执行构造器主体代码。

当然，最好精心地组织初始化代码，以便其他程序员轻松理解，而不要求他们都是语言专家。例如，如果让类的构造器依赖于实例字段声明的顺序，那就会显得很奇怪并且容易引起错误。

可以通过提供一个初始值，或者使用一个静态的初始化块来初始化静态字段。前面已经介绍过第一种机制：

```
private static int nextId = 1;
```

如果类的静态字段需要很复杂的初始化代码，那么可以使用静态的初始化块。

将代码放在一个块中，并标记关键字static。下面是一个示例。我们希望将员工ID的起始值赋为一个小于10 000的随机整数。

```
private static Random generator = new Random();
// static initialization block
static
{
   nextId = generator.nextInt(10000);
}
```

在类第一次加载的时候，会完成静态字段的初始化。与实例字段一样，除非将静态字段显式地设置成其他值，否则默认的初始值为0、false或null。所有的静态字段初始化方法以及静态初始化块都将依照类声明中出现的顺序执行。

注释：让人惊讶的是，在JDK 6之前，完全可以用Java编写一个没有main方法的“Hello, World”程序。

```
public class Hello
{
   static
   {
      System.out.println("Hello, World");
   }
}
```

当用java Hello调用这个类时，就会加载这个类，静态初始化块将会打印“Hello, World”。在此之后才会显示一个消息指出main未定义。从Java 7以后，java程序会首先检查是否有一个main方法。

这个例子使用了Random类来生成随机数。从JDK 17开始，java.util.random包提供了考虑多种因素的强算法的实现。阅读java.util.random包的API文档，其中对如何选择算法给出了建议。然后通过提供算法名来得到一个实例，如下所示：

```
RandomGenerator generator = RandomGenerator.of("L64X128MixRandom");
```

调用generator.nextInt(n)或其他RandomGenerator方法来生成随机数。(RandomGenerator是一个接口，第6章将介绍接口概念。Random类的对象可以使用所有RandomGenerator方法。)

程序清单4-5中的程序展示了本节讨论的很多特性：

- 重载构造器；
- 用this(...)调用另一个构造器；
- 无参数构造器；
- 对象初始化块；
- 静态初始化块；
- 实例字段初始化。

程序清单4-5 ConstructorTest/ConstructorTest.java

```
1 import java.util.*;
2
3 /**
4  * This program demonstrates object construction.
5  * @version 1.02 2018-04-10
```

```
 * @author Cay Horstmann
 */
public class ConstructorTest
{
   public static void main(String[] args)
   {
      // fill the staff array with three Employee objects
      var staff = new Employee[3];

      staff[0] = new Employee("Harry", 40000);
      staff[1] = new Employee(60000);
      staff[2] = new Employee();

      // print out information about all Employee objects
      for (Employee e : staff)
         System.out.println("name=" + e.getName() + ",id=" + e.getId() + ",salary="
            + e.getSalary());
   }
}

class Employee
{
   private static int nextId;

   private int id;
   private String name = ""; // instance field initialization
   private double salary;

   private static Random generator = new Random();

   // static initialization block
   static
   {
      // set nextId to a random number between 0 and 9999
      nextId = generator.nextInt(10000);
   }

   // object initialization block
   {
      id = nextId;
      nextId++;
   }

   // three overloaded constructors
   public Employee(String n, double s)
   {
      name = n;
      salary = s;
   }

   public Employee(double s)
   {
      // calls the Employee(String, double) constructor
      this("Employee #" + nextId, s);
   }
```

```
61
62    // the default constructor
63    public Employee()
64    {
65       // name initialized to ""--see above
66       // salary not explicitly set--initialized to 0
67       // id initialized in initialization block
68    }
69
70    public String getName()
71    {
72       return name;
73    }
74
75    public double getSalary()
76    {
77       return salary;
78    }
79
80    public int getId()
81    {
82       return id;
83    }
84 }
```

API java.util.Random 1.0

- Random()
  构造一个新的随机数生成器。

API java.util.random.RandomGenerator 17

- int nextInt(int n)
  返回一个 0 ～ n-1 之间的随机数。
- static RandomGenerator of(String name)
  由给定算法名生成一个随机数生成器。算法“L64X128MixRandom”对大多数应用都适用。

4.6.8 对象析构与 finalize 方法

有些面向对象的程序设计语言（特别是 C++）有显式的析构器方法，其中放置一些清理代码，当对象不再使用可能需要执行这些清理代码。在析构器中，最常见的操作是回收分配给对象的存储空间。由于 Java 会完成自动的垃圾回收，不需要人工回收内存，所以 Java 不支持析构器。

当然，某些对象使用了内存之外的其他资源，例如，文件或使用系统资源的另一个对象的句柄。在这种情况下，当资源不再需要时，将其回收和再利用就十分重要。

如果一个资源一旦使用完就需要立即关闭，那么应当提供一个 close 方法来完成必要的清理工作。可以在对象使用完时调用这个 close 方法。第 7 章将介绍如何确保自动调用这个方法。

如果可以等到虚拟机退出，那么可以用方法 Runtime.addShutdownHook 增加一个“关闭钩”（shutdown hook）。在 Java 9 中，可以使用 Cleaner 类注册一个动作，当对象不再可达时（除了清洁器还能访问，其他对象都无法访问这个对象），就会完成这个动作。在实际中这些情况很少见。可以参见 API 文档来了解这两种方法的详细内容。

警告：不要使用 finalize 方法来完成清理。这个方法原本要在垃圾回收器清理对象之前调用。不过，你并不能知道这个方法到底什么时候调用，而且该方法已经被废弃。